CS 5402: Introduction to Data Mining

SP2026

(Lecture - 16)

Convolutional Neural Networks (CNN)

Dr. Huiyuan Yang

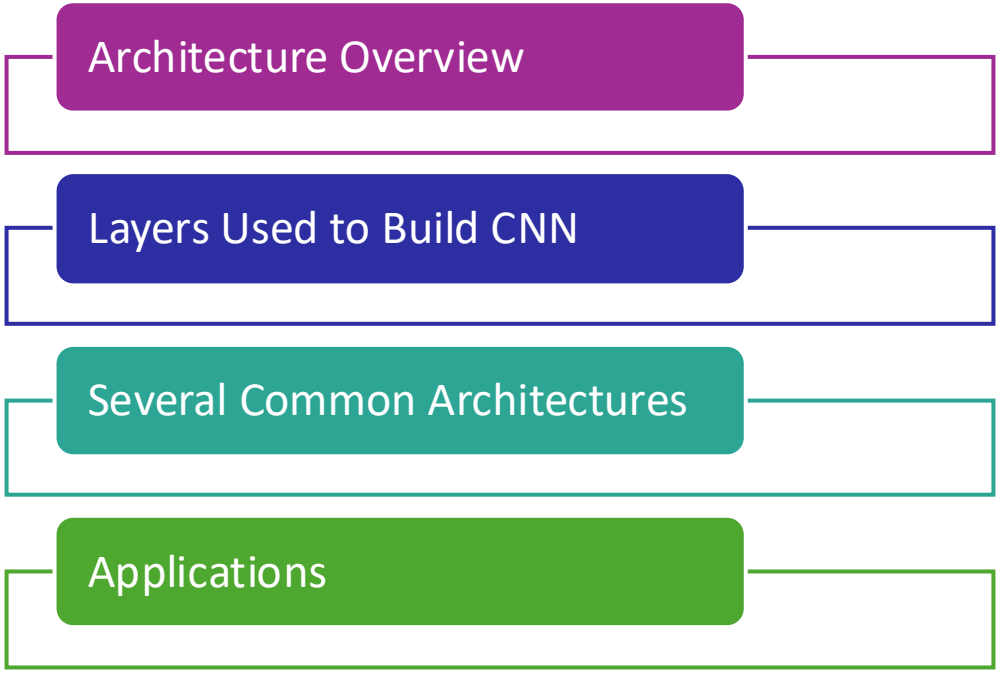
Department of Computer Science
Missouri S&T

https://hyang428.github.io

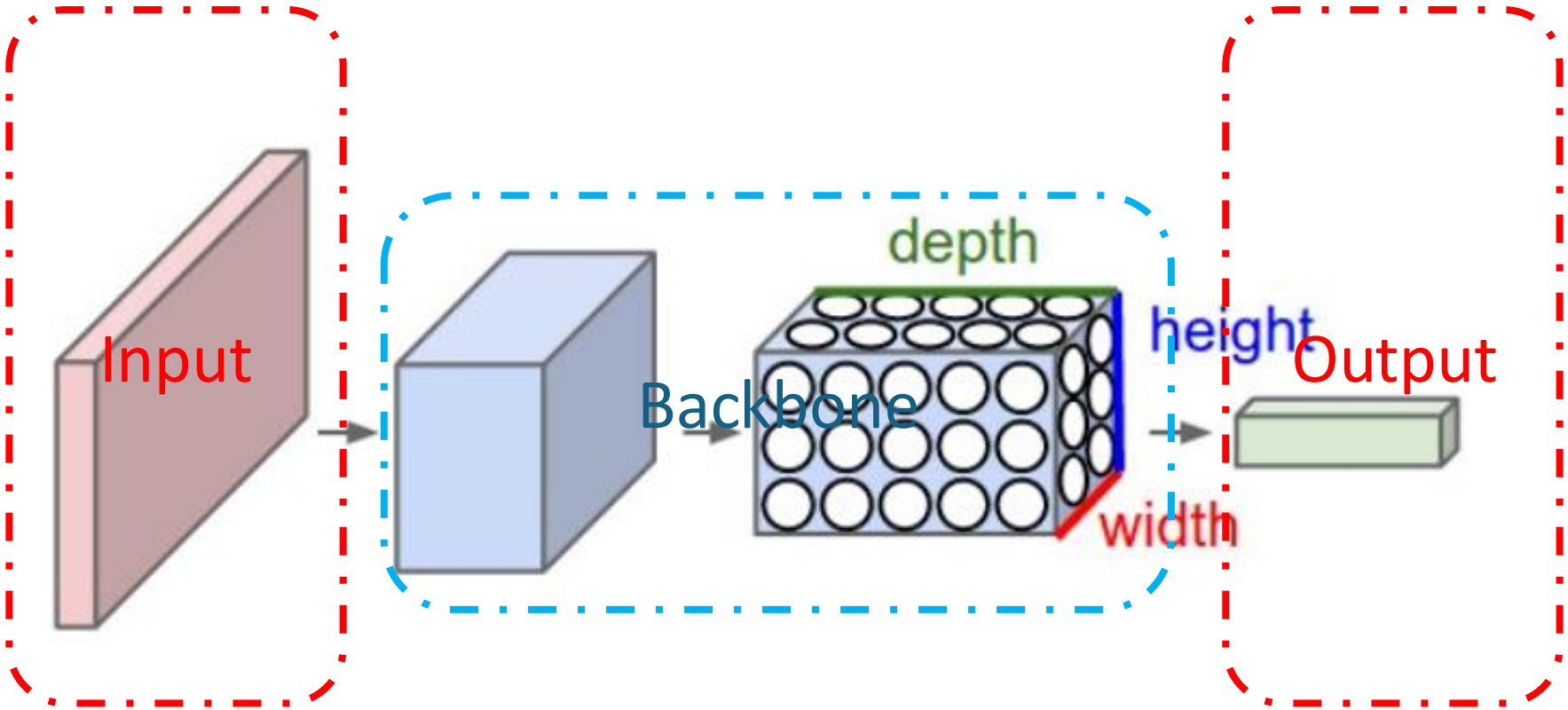
MISSOURI S&T

Introduction to Convolutional Neural Networks and its Application

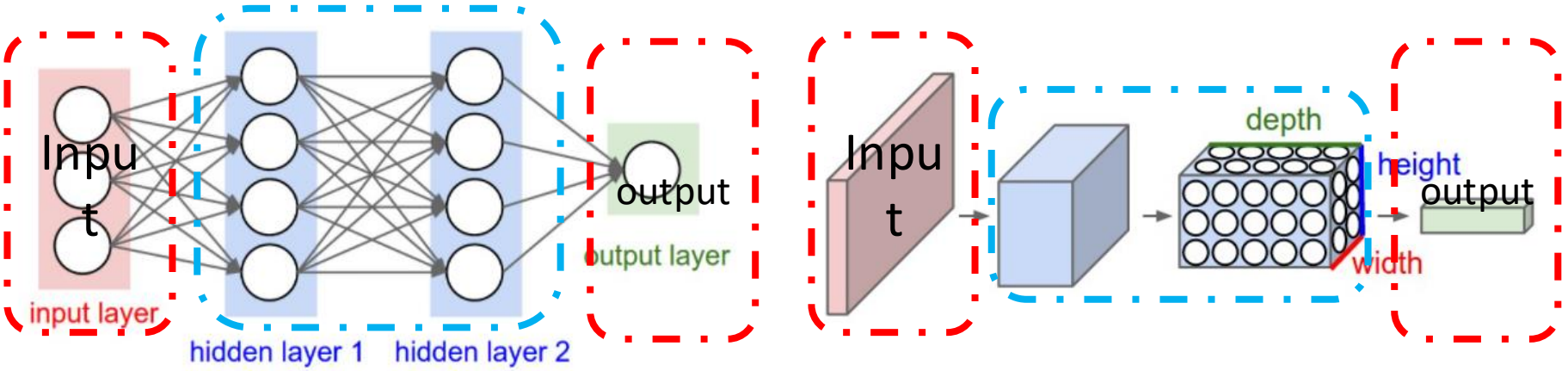
Today's Agenda



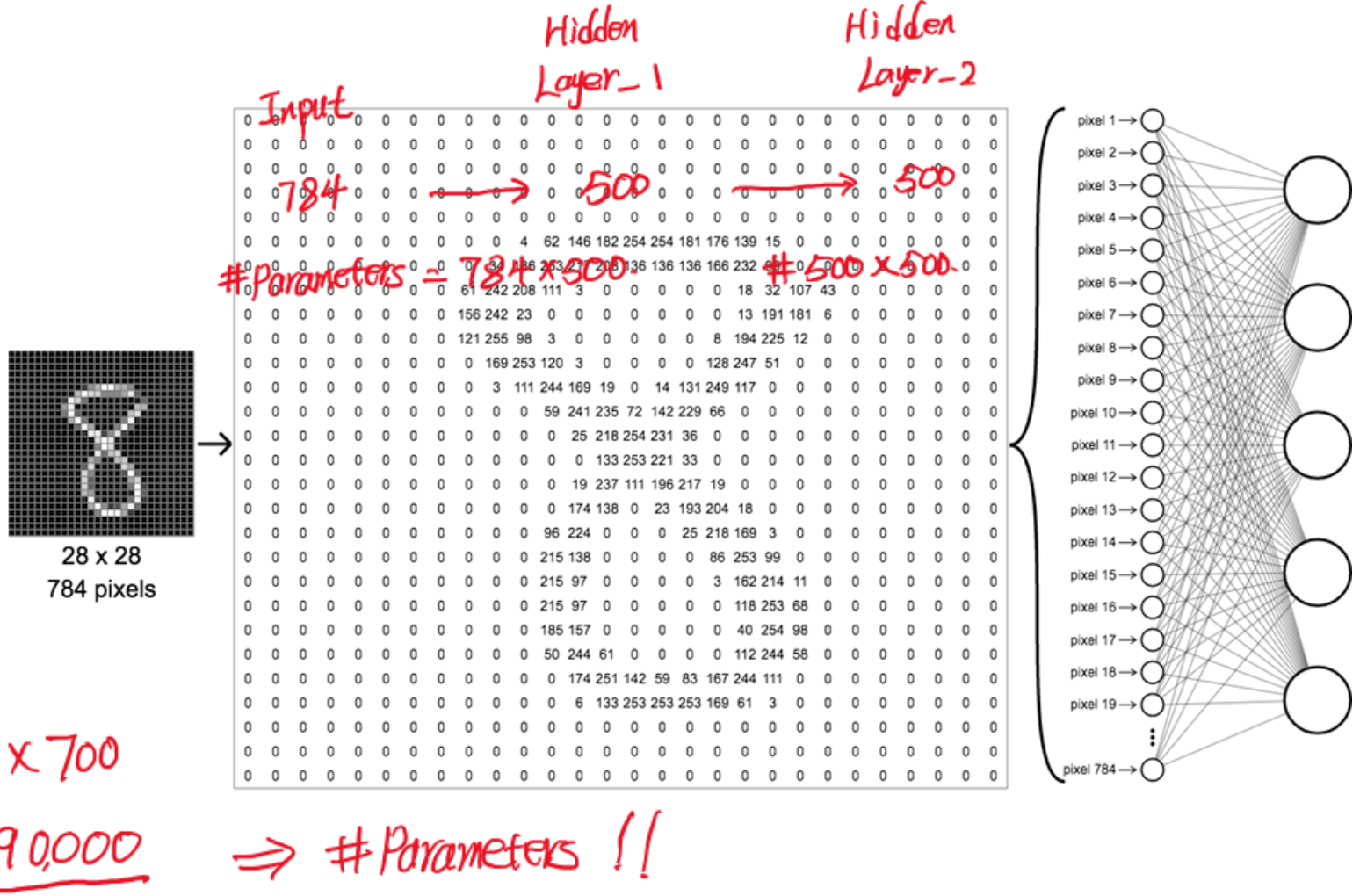
Architecture Overview



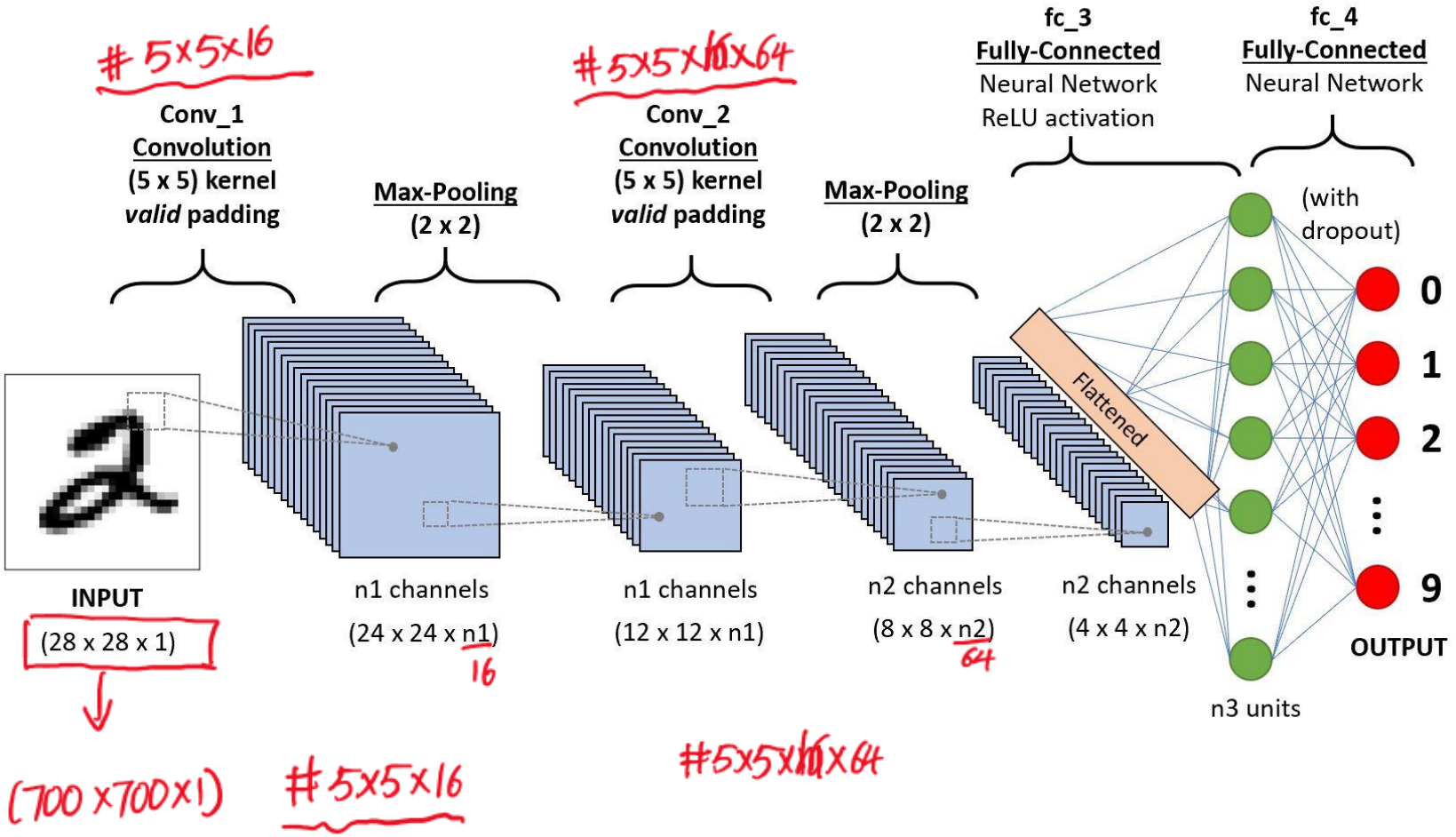
Architecture Overview



Why not just use standard Neural Networks?

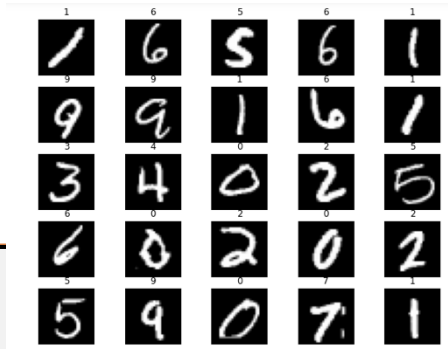


The Solution: Convolution



Layers Used to Build CNN

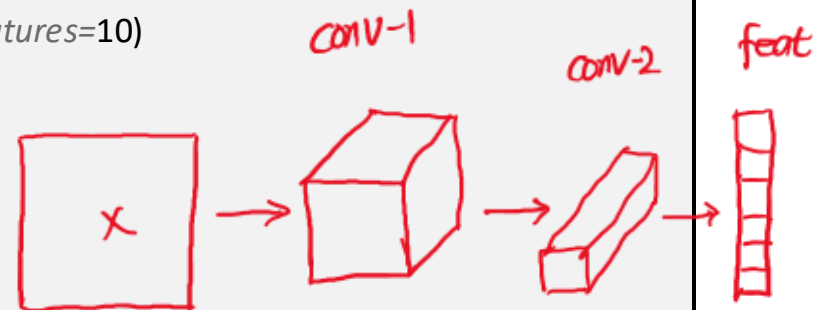
- Convolutional Layer
- Activation Layer
- Normalization Layer
- Pooling Layer
- Fully Connected Layer



```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=5, stride=1, padding=2)
        self.bn1 = nn.BatchNorm2d(num_features=16)
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=5, stride=1, padding=2)
        self.bn2 = nn.BatchNorm2d(num_features=32)
        self.fc1 = nn.Linear(in_features=32 * 7 * 7, out_features=10)
        self.pool = nn.MaxPool2d(kernel_size=2)
        self.relu = nn.ReLU()

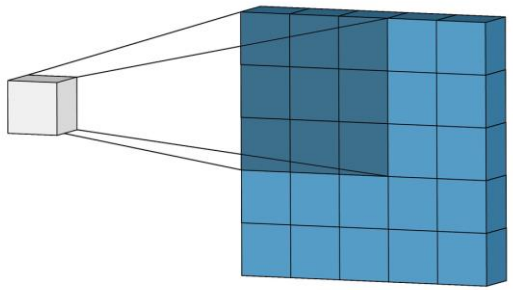

    def forward(self, x):
        x = self.pool( self.relu( self.bn1( self.conv1(x) ) ) )
        x = self.pool( self.relu( self.bn2( self.conv2(x) ) ) )

        # flatten the output of conv2 to (batch_size, 32 * 7 * 7)
        x = x.view(x.size(0), -1)
        x = self.fc1(x)
        return x
```

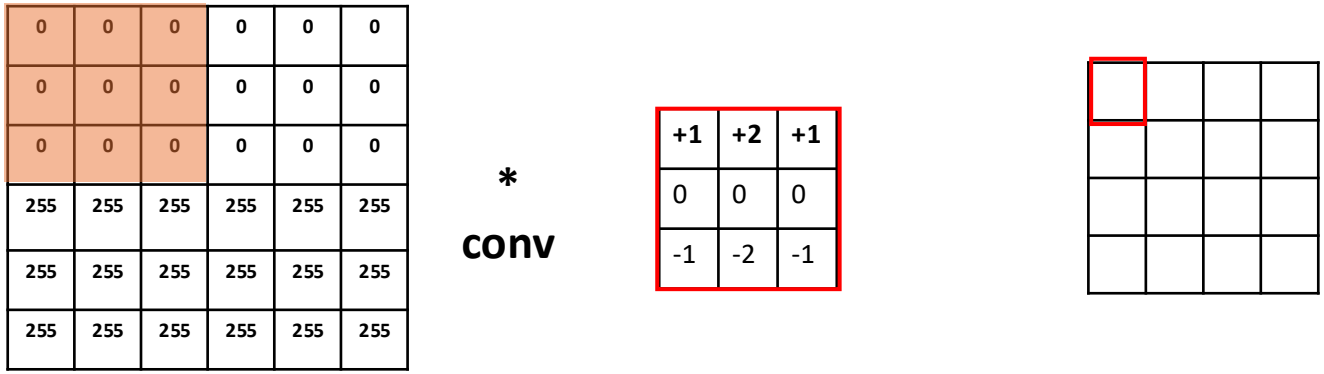


Convolutional Layer

- nn.Conv2d(*in_channels=**, *out_channels=**, *kernel_size=*3, stride=1, padding=0)



Convolutional Layer



Kernel/filter/operator

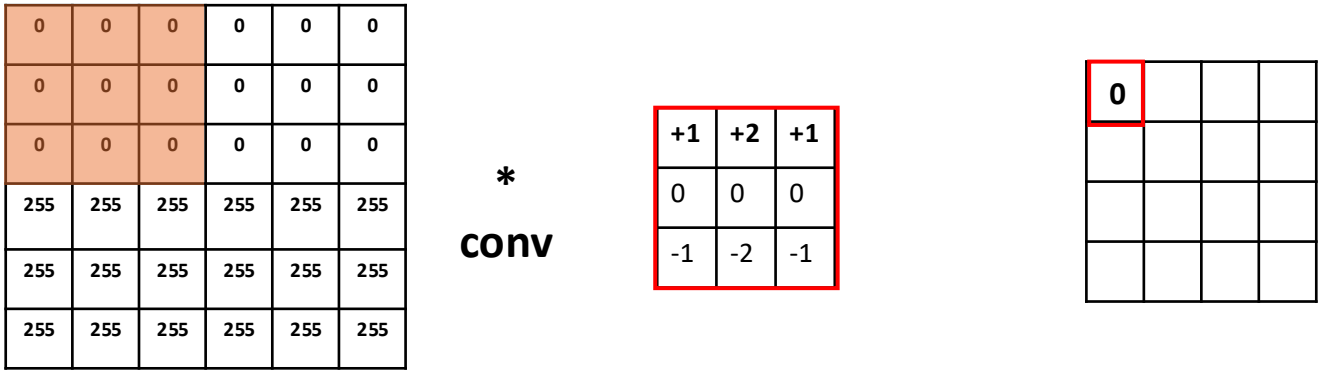
$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{3}\sum_{j=1}^{3} w_{ij} \times x_{ij} = y$$

Element-wise multiplication + addition

0 x 1 + 0 x 2 + 0 x 1 + 0 x 0 + 0x(-1)+ 0x(-2) 0x(-1) = 0

Convolutional Layer



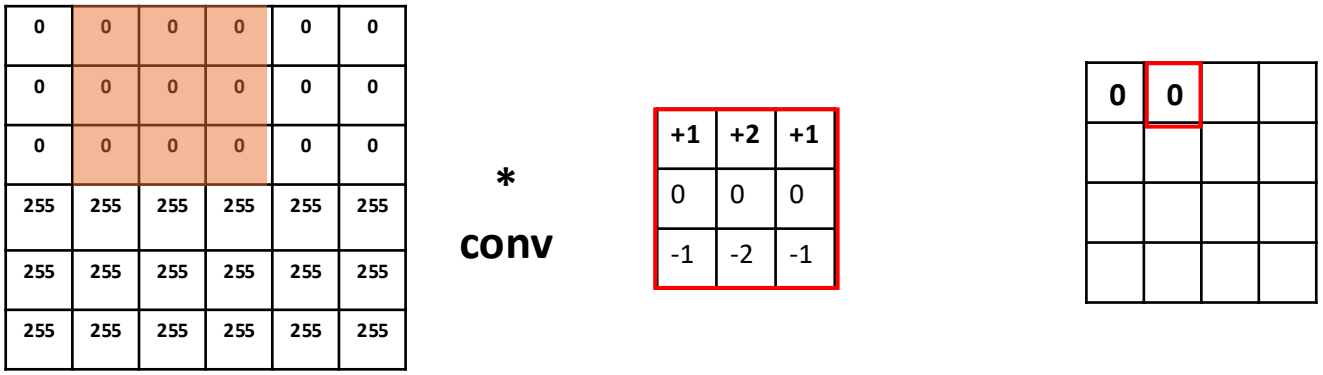
Kernel/filter/operator

$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{3}\sum_{j=1}^{3} w_{ij} \times x_{ij} = y$$

0 x 1 + 0 x 2 + 0 x 1 + 0 x 0 + 0x(-1)+ 0x(-2) 0x(-1) = 0

Convolutional Layer



Kernel/filter/operator

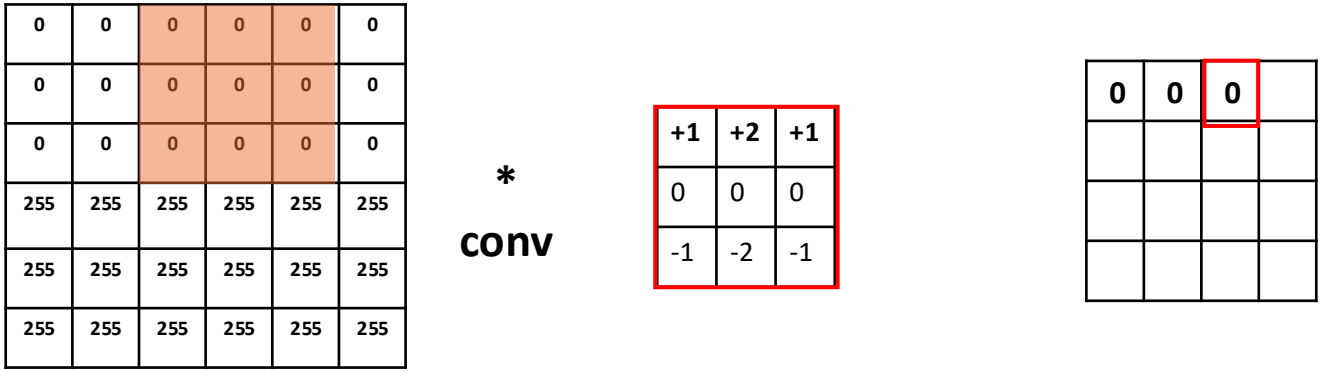
$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{3}\sum_{j=1}^{3} w_{ij} \times x_{ij} = y$$

Stride = 1
Filter shifts by 1

0 x 1 + 0 x 2 + 0 x 1 + 0 x 0 + 0x(-1)+ 0x(-2) 0x(-1) = 0

Convolutional Layer

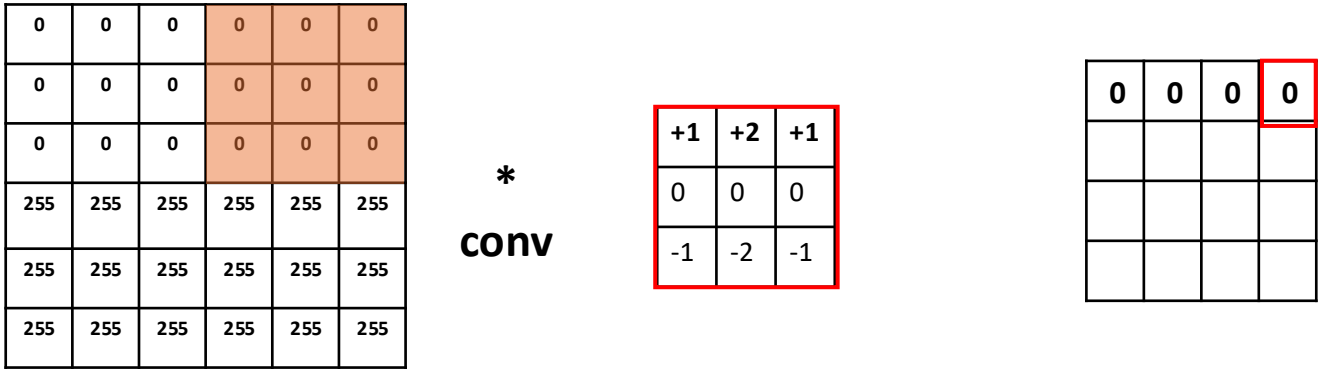


Kernel/filter/operator

$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{3}\sum_{j=1}^{3} w_{ij} \times x_{ij} = y$$

Convolutional Layer

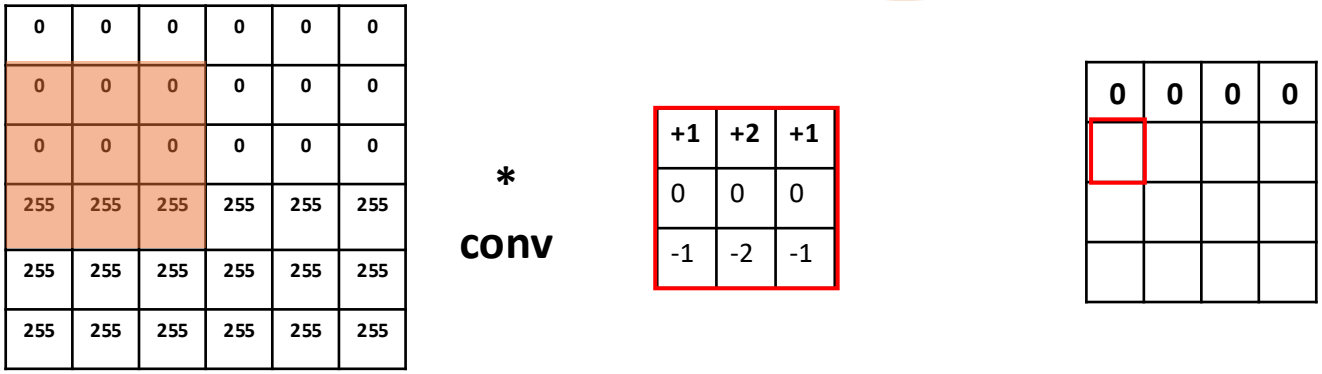


Kernel/filter/operator

$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{3}\sum_{j=1}^{3} w_{ij} \times x_{ij} = y$$

Convolutional Layer

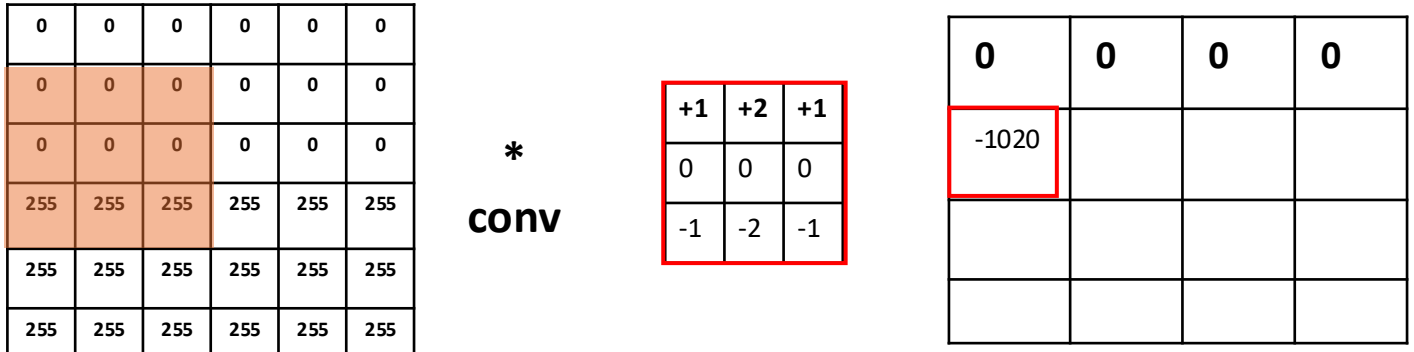


Kernel/filter/operator

$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{3}\sum_{j=1}^{3} w_{ij} \times x_{ij} = y$$

Convolutional Layer



x_{ij} w_{ij} $= y_{kl}$

$$\sum_{i=1}^{3}\sum_{j=1}^{3} w_{ij} \times x_{ij} = y$$

255x(-1) + 255 x (-2) + 255 x (-1)= = -1020

Convolutional Layer

0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
255	255	255	255	255	255
255	255	255	255	255	255
255	255	255	255	255	255

Input 5x5

*

conv

+1	+2	+1
0	0	0
-1	-2	-1

Filter 3x3

0	0	0	0
-1020	-1020	-1020	-1020
-1020	-1020	-1020	-1020
0	0	0	0

Output 4x4

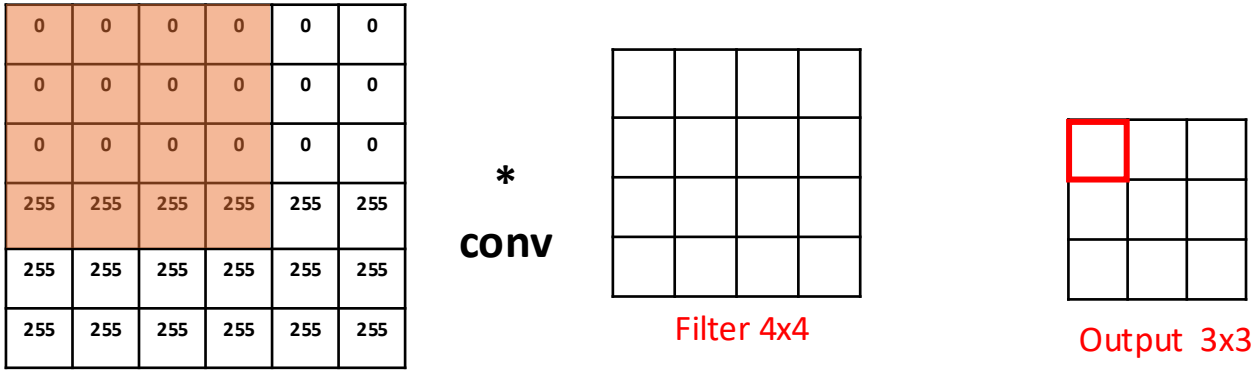
Kernel/filter/operator

$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{3}\sum_{j=1}^{3} w_{ij} \times x_{ij} = y$$

- filter size.
- stride.

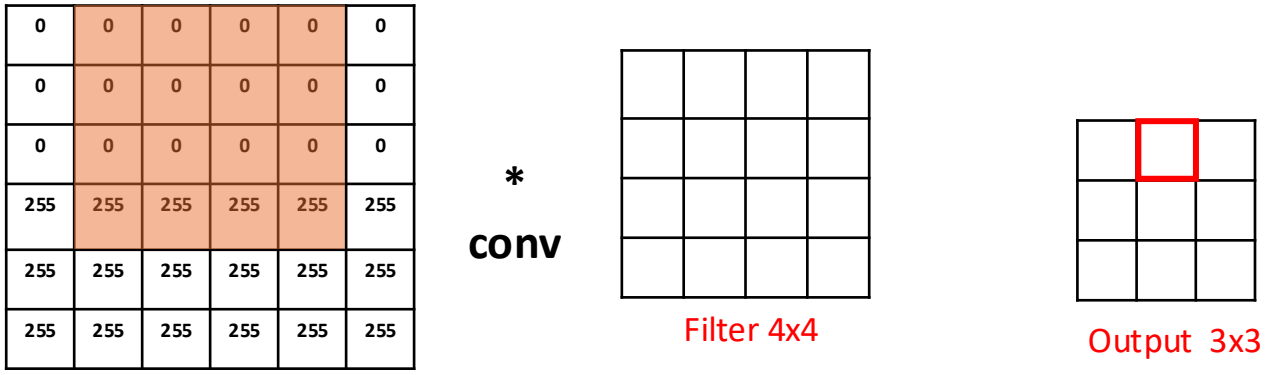
Convolutional Layer- filter size



$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{4}\sum_{j=1}^{4} w_{ij} \times x_{ij} = y$$

Convolutional Layer- filter size



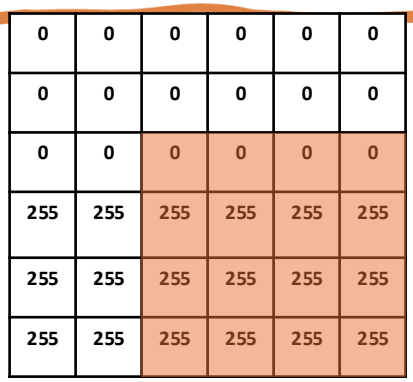
Input 5x5

Kernel/filter/operator

$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{4}\sum_{j=1}^{4} w_{ij} \times x_{ij} = y$$

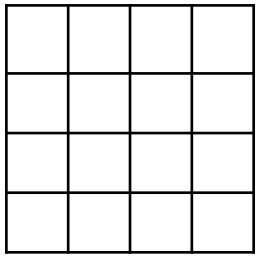
Convolutional Layer- filter size



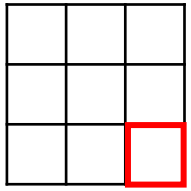
Input 5x5

*

conv



Filter 4x4



Output 3x3

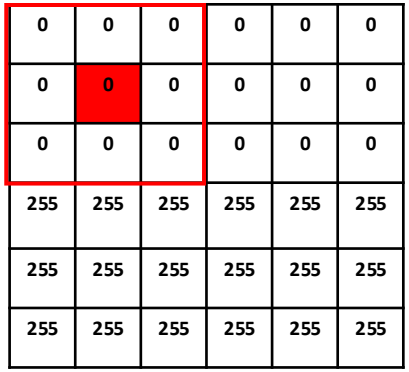
Kernel/filter/operator

dimensions of the output changed!

$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{4}\sum_{j=1}^{4} w_{ij} \times x_{ij} = y$$

Convolutional Layer - Stride



*

conv

+1	+2	+1
0	0	0
-1	-2	-1

Kernel/filter/operator

0	0

stride=3

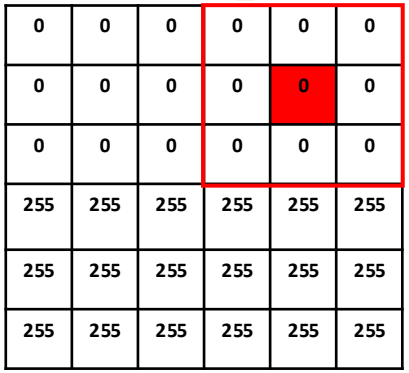
$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{3}\sum_{j=1}^{3} w_{ij} \times x_{ij} = y$$

0 x 1 + 0 x 2 + 0 x 1 + 0 x 0 + 0x(-1)+ 0x(-2) 0x(-1) = 0

nn.Conv2d(*in_channels=*, out_channels=*, kernel_size=3*, stride=3, padding=0)

Convolutional Layer - Stride



*

conv

+1	+2	+1
0	0	0
-1	-2	-1

Kernel/filter/operator

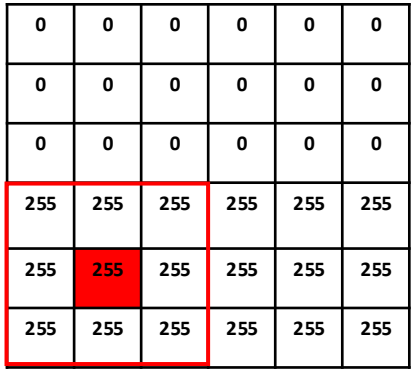
0	0

stride=3

$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{3}\sum_{j=1}^{3} w_{ij} \times x_{ij} = y$$

Convolutional Layer - Stride



*

conv

+1	+2	+1
0	0	0
-1	-2	-1

Kernel/filter/operator

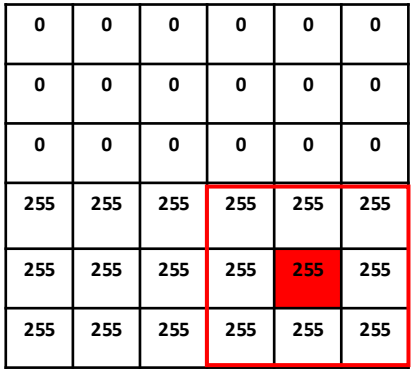
0	0
0	

stride=3

$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{3}\sum_{j=1}^{3} w_{ij} \times x_{ij} = y$$

Convolutional Layer - Stride



*

conv

+1	+2	+1
0	0	0
-1	-2	-1

Kernel/filter/operator

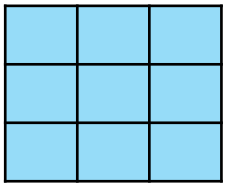
0	0
0	0

stride=3

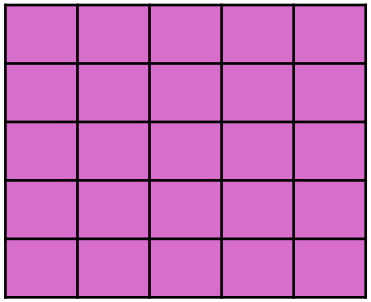
$$x_{ij} \qquad w_{ij} \qquad = y_{kl}$$

$$\sum_{i=1}^{3}\sum_{j=1}^{3} w_{ij} \times x_{ij} = y$$

Convolutional Layer - Padding



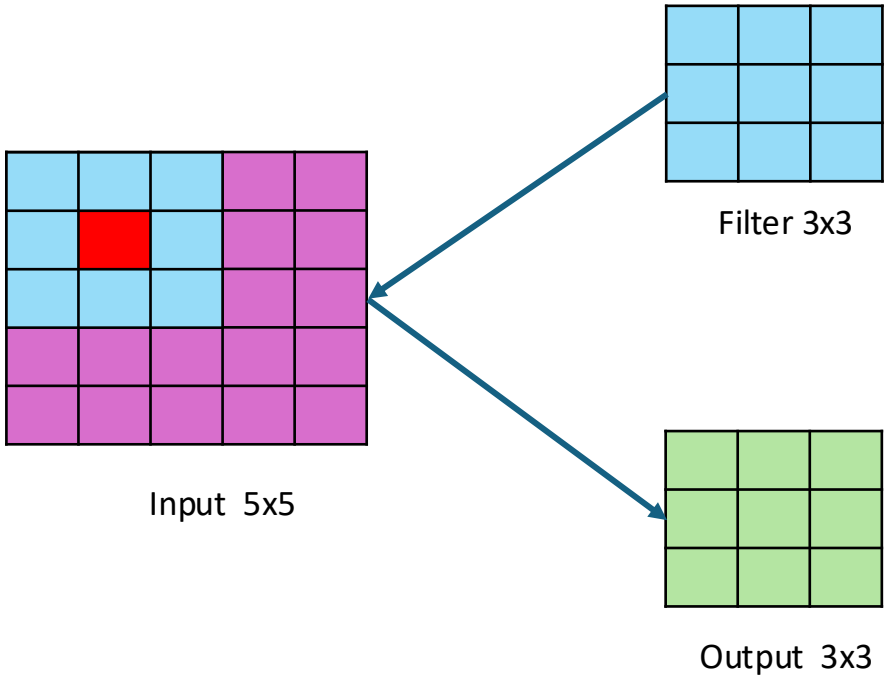
Filter 3x3



Input 5x5

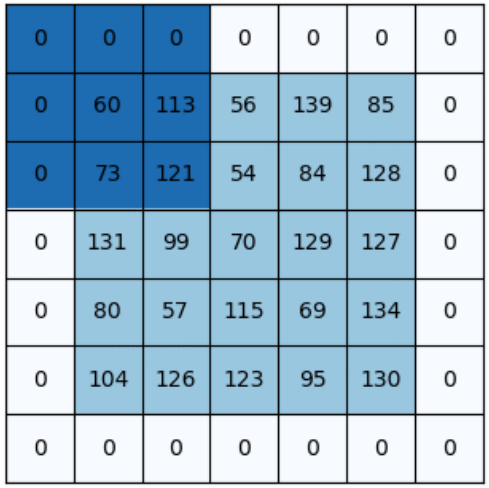
nn.Conv2d(*in_channels=**, *out_channels=**, *kernel_size=5*, stride=1, padding=0)

Convolutional Layer - Padding

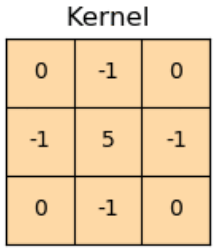


nn.Conv2d(*in_channels=**, *out_channels=**, *kernel_size=5*, stride=1, padding=0)

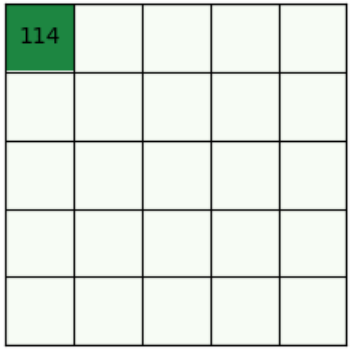
Convolutional Layer - Padding



Input 5x5



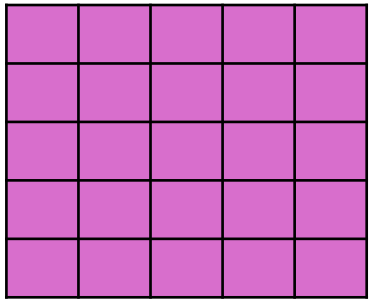
Filter 3x3



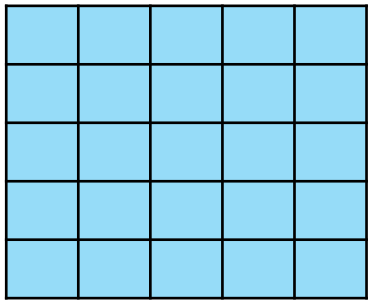
Output 5x5

nn.Conv2d(*in_channels=**, *out_channels=**, *kernel_size=3*, stride=1, padding=1)

Convolutional Layer - Padding



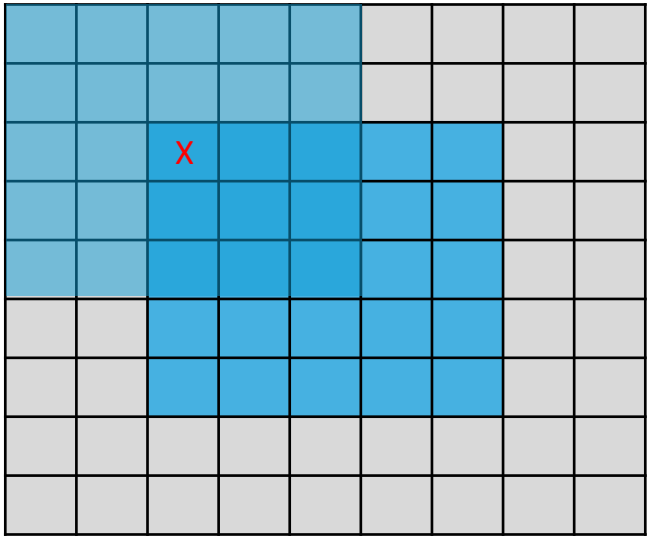
Input 5x5



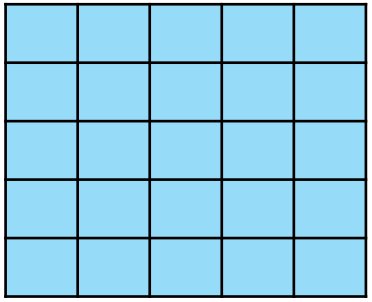
Filter 5x5

nn.Conv2d(*in_channels=**, *out_channels=**, *kernel_size=5*, stride=1, padding=2)

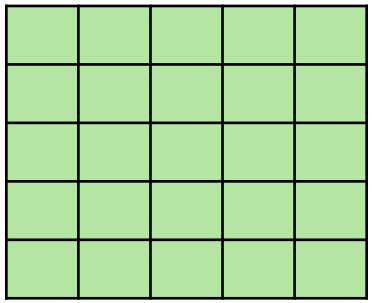
Convolutional Layer - Padding



Input 5x5



Filter 5x5



Output 5x5

nn.Conv2d(*in_channels=**, *out_channels=**, *kernel_size=5*, stride=1, padding=2)

Normalization Layer and Activation Layer

- Batch Normalization:

$$\mu = \frac{1}{m}\sum_{i=1}^{m} x_i$$

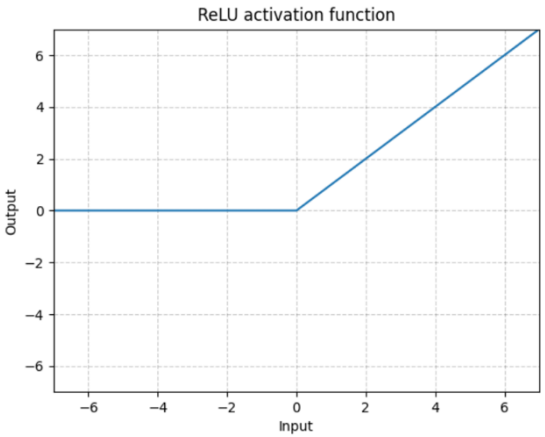
[scale] [shift.] Learnable parameters.

$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta$$

$$\frac{1}{m}\sum_{i=1}^{m} (x_i - \mu)^2$$

- Activation Function:

$$\mathrm{ReLU}(x) = (x)^+ = \max(0, x)$$



Pooling

- It's used to reduce the spatial size of the representation.
- Therefore, reduce the amount of parameters and computation in the network.
- Most common form:
  - Max pooling
  - Average pooling

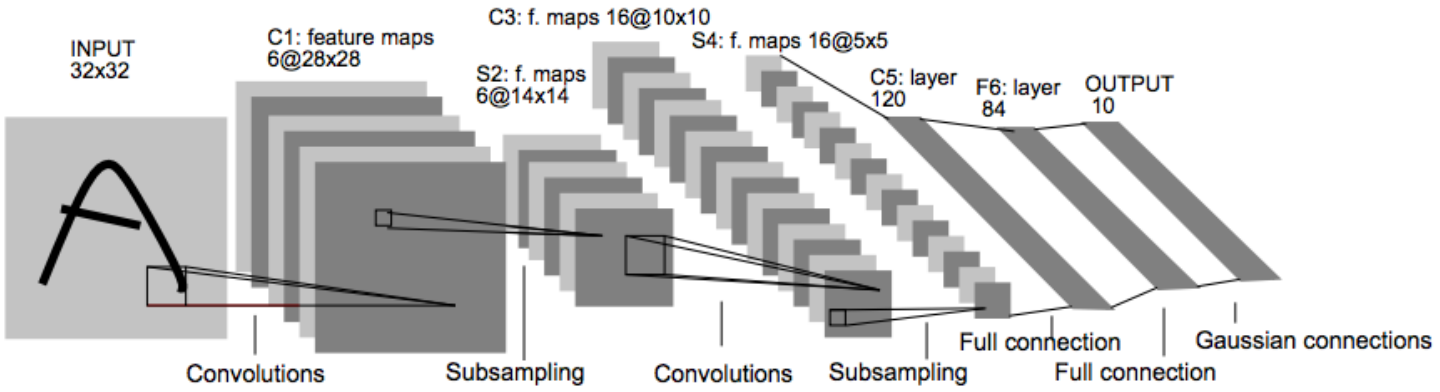
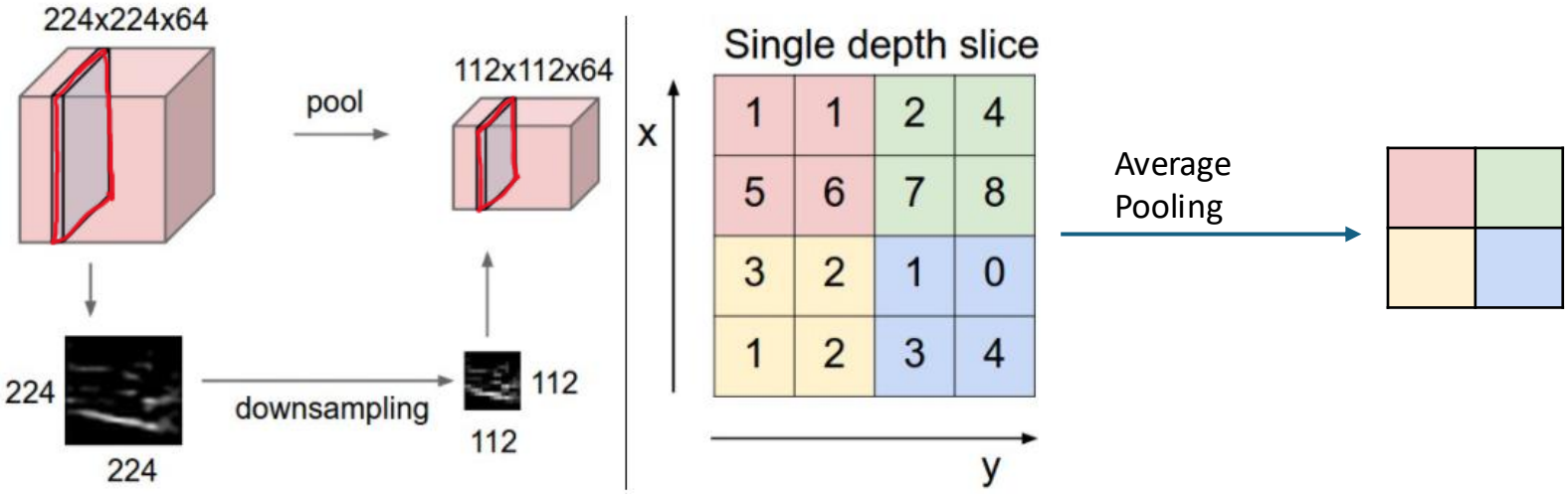


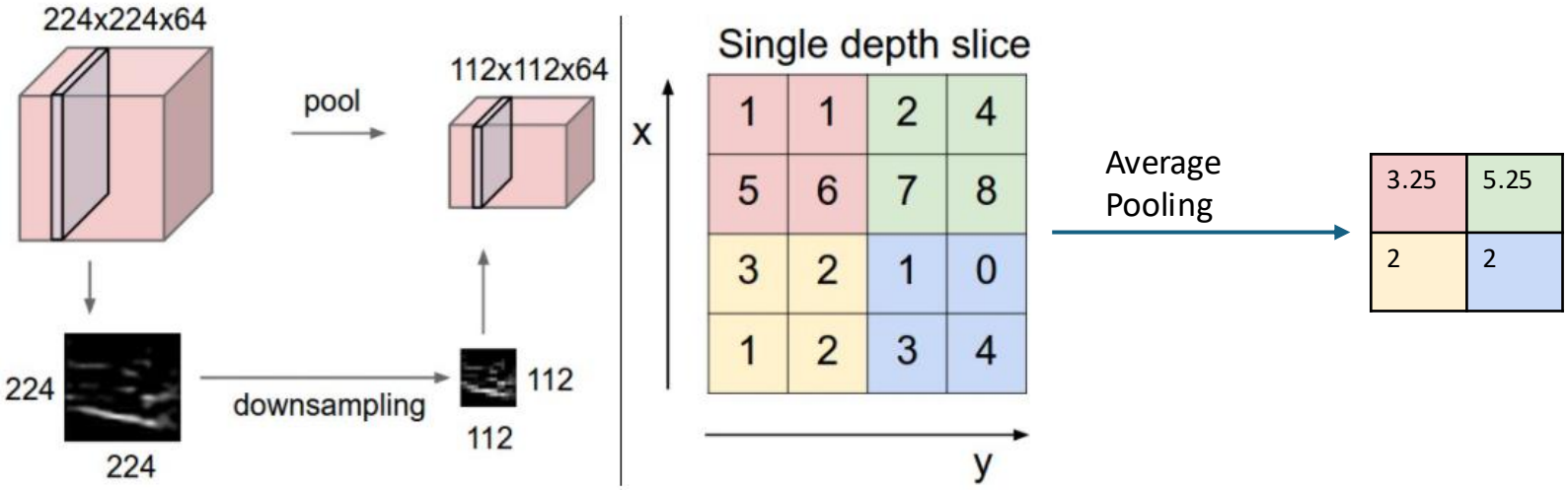
Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

Pooling



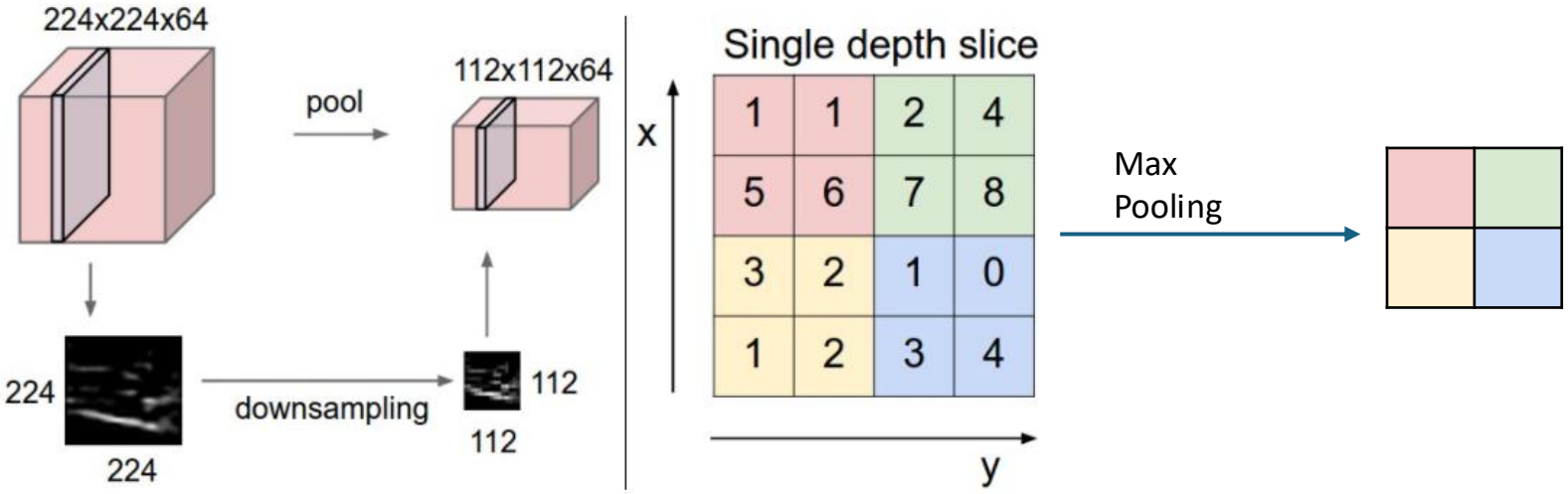
https://cs231n.github.io/convolutional-networks/

Pooling



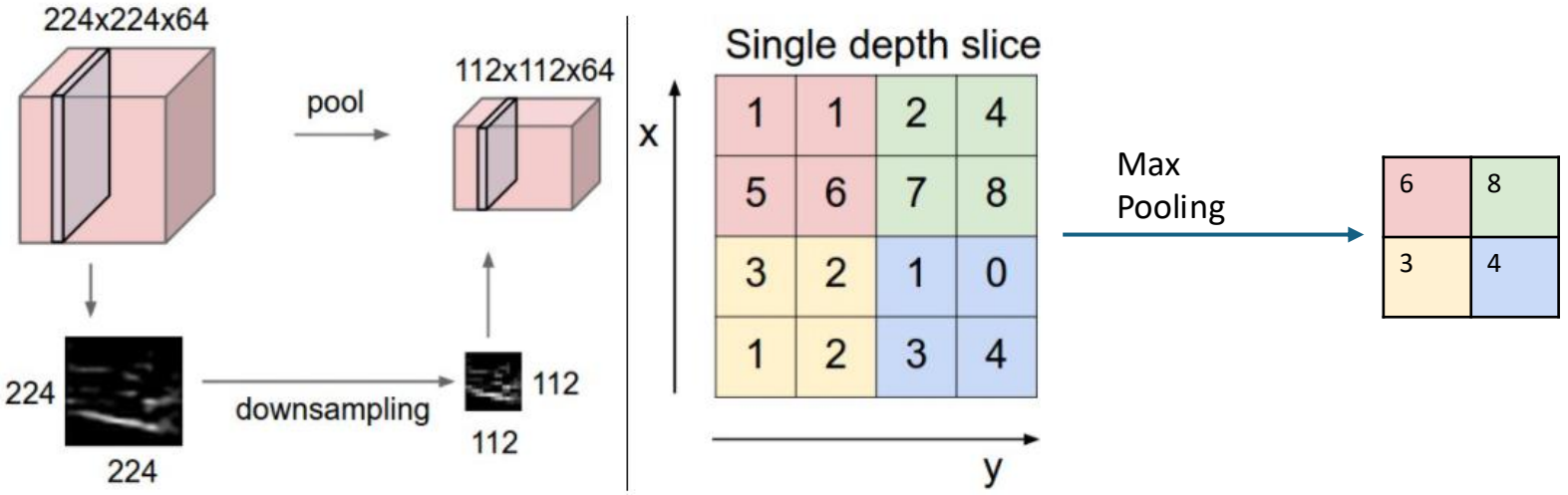
https://cs231n.github.io/convolutional-networks/

Pooling



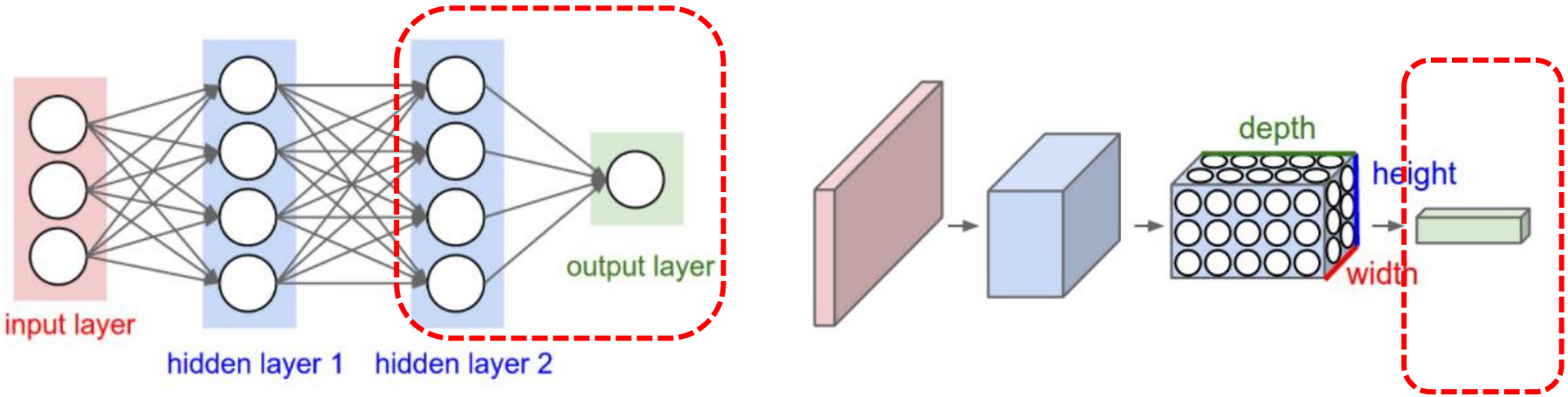
https://cs231n.github.io/convolutional-networks/

Pooling



https://cs231n.github.io/convolutional-networks/

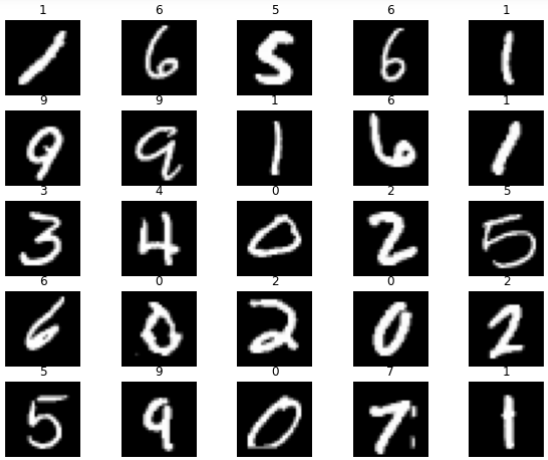
Fully Connected Layer



```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=5, stride=1, padding=2)
        self.bn1 = nn.BatchNorm2d(num_features=16)
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=5, stride=1, padding=2)
        self.bn2 = nn.BatchNorm2d(num_features=32)
        self.fc1 = nn.Linear(in_features=7 * 7 *32, out_features=10)
        self.pool = nn.MaxPool2d(kernel_size=2)
        self.relu = nn.ReLU()


    def forward(self, x):
        x = self.pool(self.relu(self.bn1(self.conv1(x))))
        x = self.pool(self.relu(self.bn2(self.conv2(x))))

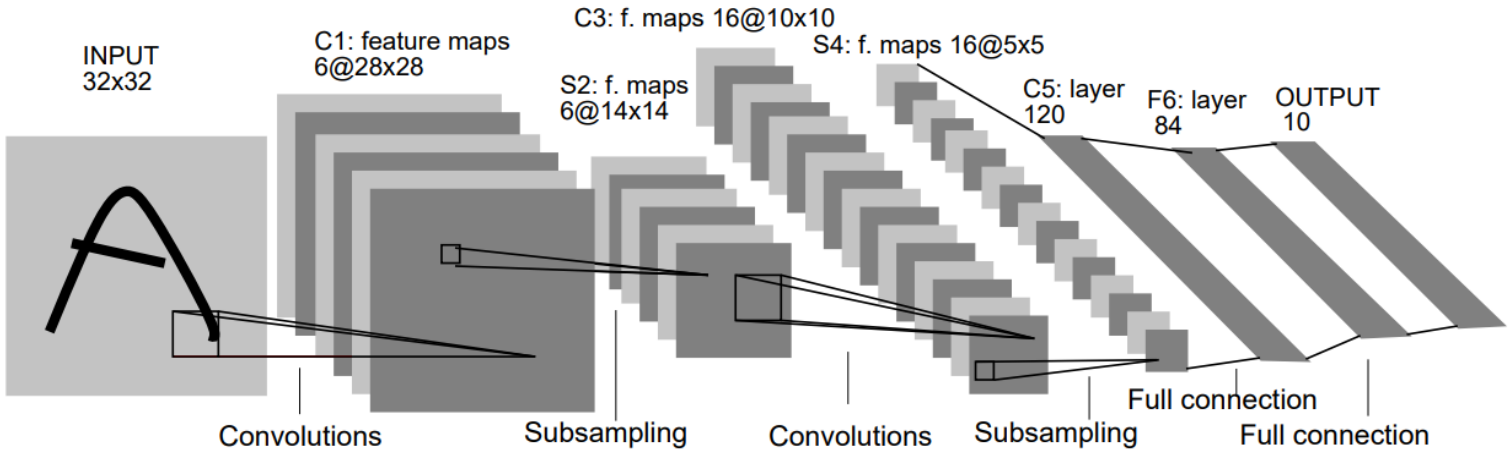
        # flatten the output of conv2 to (batch_size, 32 * 7 * 7)
        x = x.view(x.size(0), -1)
        x = self.fc1(x)
        return x
```



Several Common Architectures

- LeNet
- AlexNet
- GoogLeNet
- VGG
- ResNet

LeNet (1998)



- The first successful applications of CNN, was developed by **Yann LeCun** in 1998, and was used to read Zip codes, digits, etc.
- The first application of the backpropagation algorithm to practical application.
- It means the emergence of CNN and defines the basic component of CNN, but it was not popular at that time because of the lack of hardware equipment (especially GPU) and large labeled dataset.

The “AI Winter”

Data



- Imagenet, biggest dataset for object classification, first published in **2009**.
- 1000 classes, 1.2M training images, 150K for test

Hardware



2008, GeForce 200 Series, ~1GB, 240 CUDA Cores, 933 GFLOPS

2022, GeForce 40 Series, 8~16GB, 18,432 CUDA Cores, 40,000 GFLOPS

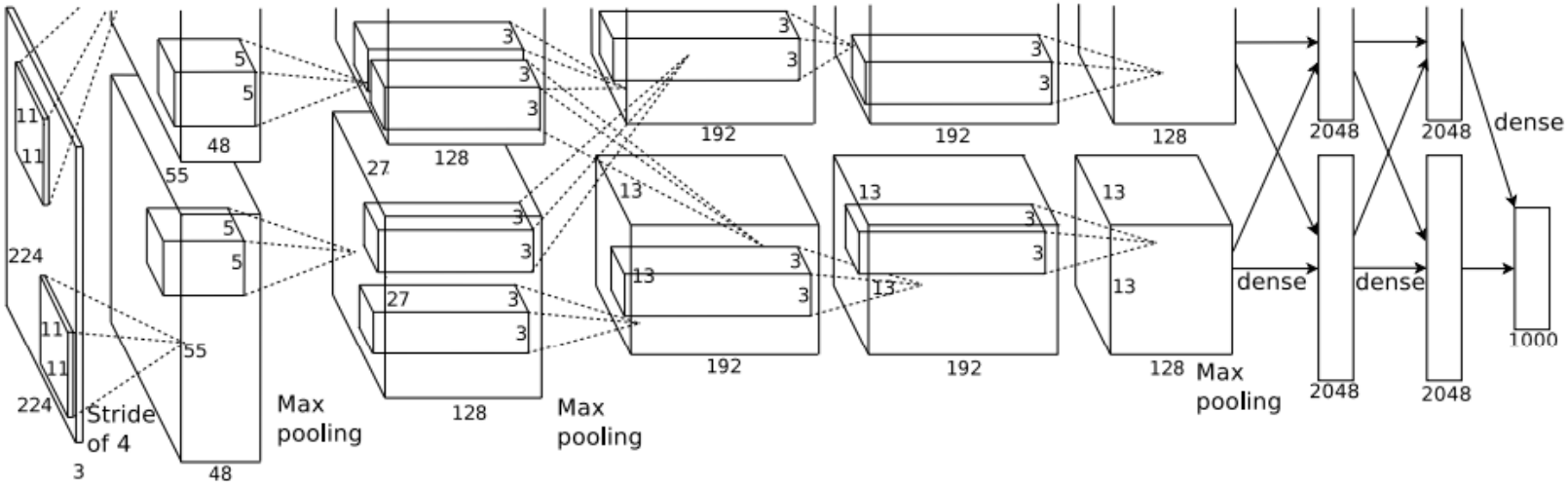
2023, NVIDIA H100 GPU, 80GB, 16,896 (SXM5), 3,958,000 GFLOPS

2024, NVIDIA H200 GPU, 141GB, 16,896 (SXM5), 7,916,000 GFLOPS

2022, 45 terabytes of text data used to train GPT-3.5

2023, X22 -> GPT-4

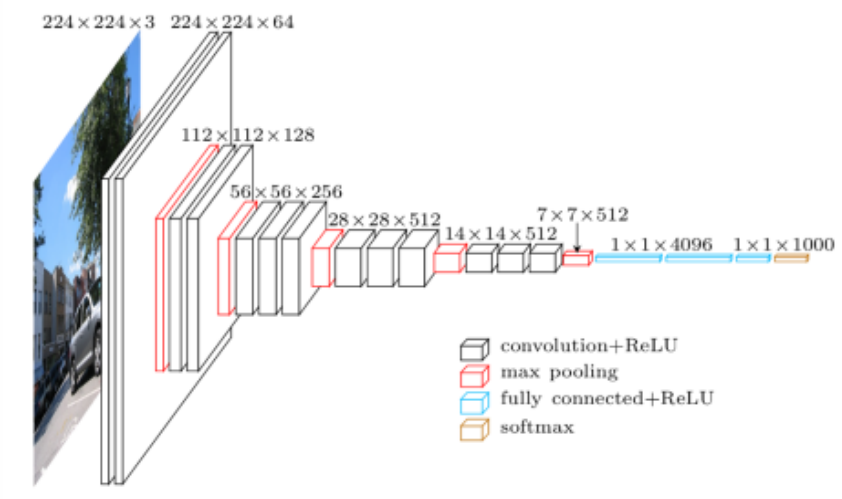
AlexNet (2012)



- The first popular convolutional networks in computer vision, was developed by Alex Krizhevsky, Ilya Sutskever (co-founder and CTO of OpenAI) and Geoffrey Hinton (Turing Award 2019, and Nobel Prize, 2024) in 2012.
- Trained and tested on ImageNet: 1.3 million high-resolution images with 1,000 different classes, and significantly outperformed the previous best performance (top 5 error was reduced from 26% to 16%)
- AlexNet had a similar architecture to LeNet, but was deeper and bigger, and also very efficient GPU implementation of convolutional nets.

Historical note: Trained on GTX 580 GPU with only 3 GB of memory.
Network spread across 2 GPUs, half the neurons (feature maps) on each GPU.

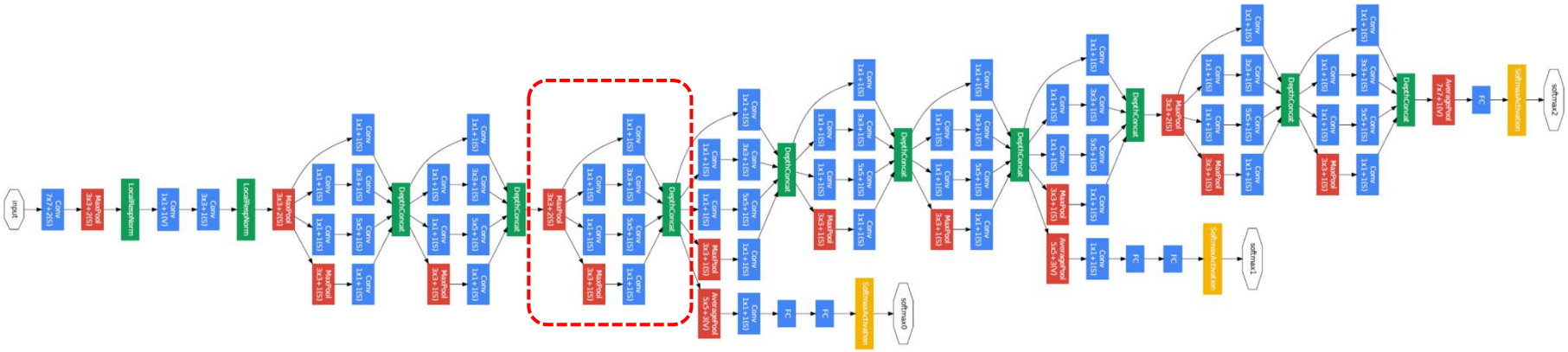
VGG (2014)



- Small filters (only 3x3 CONV), deeper networks (16-19 layers).
- 7.3% top 5 error in ILSVRC'14
- Another popular model in the early-stage, as their pretrained model is available for plug and paly use in **Caffe**..
- Main contribution was in showing that the depth of the network is a critical component for good performance.

[Simonyan and Zisserman, 2014]

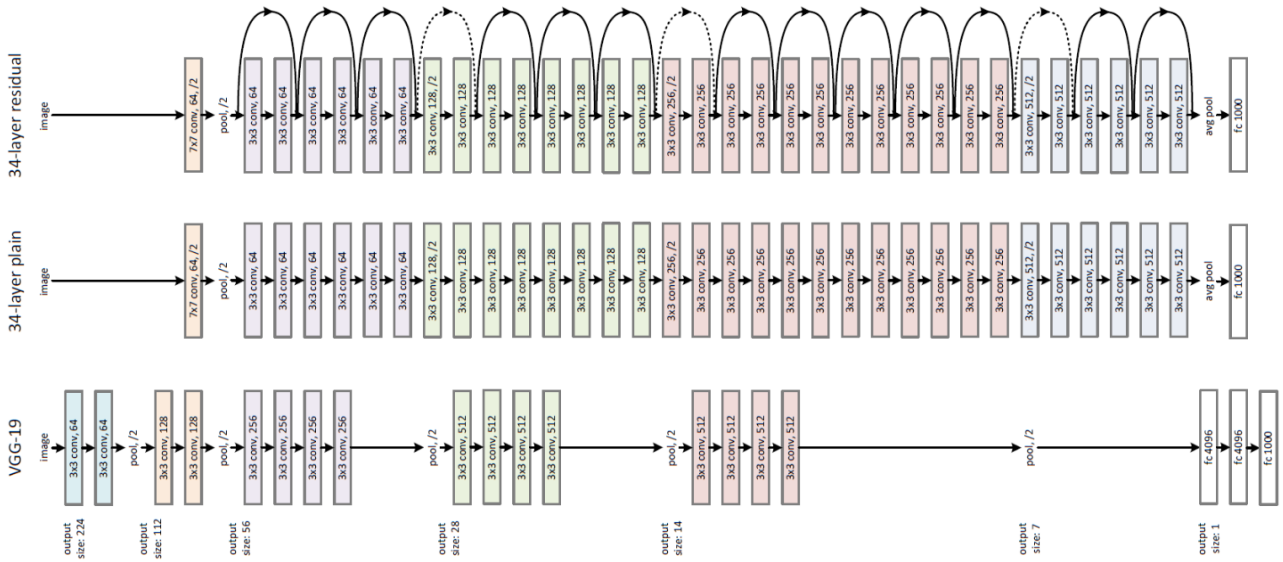
GoogLeNet (2014)

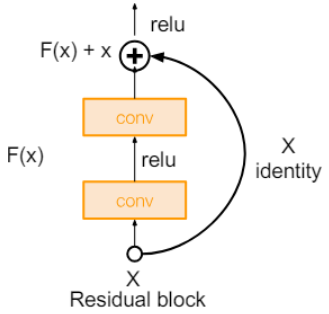


- Winner of the ImageNet Large Scale Visual Recognition Challenge 2014 (ILSVRC2014).
- Main contribution was the development of an *Inception Module* that dramatically reduced the number of parameters in the network
  - Only 5 million parameters!
  - 60 million parameters in AlexNet.
  - 138 million parameters in VGG
- There are also several follow-up versions to the GoogLeNet, most recently Inception-v4.

[Szegedy et al., 2014]

ResNet (2015)





- Very deep networks using residual connections.
- 152-layer model for ImageNet
- ILSVRC'15 classification winner (3.57% top 5 error)
- ResNets are the most popular models and are the default choice for using CNN in practice, such as ResNet18, ResNet34, ResNet50, ResNet101, ResNet152.
- It features special skip connections and a heavy use of batch normalization.

[He et al., 2015]

The CNN Architectures Hall

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

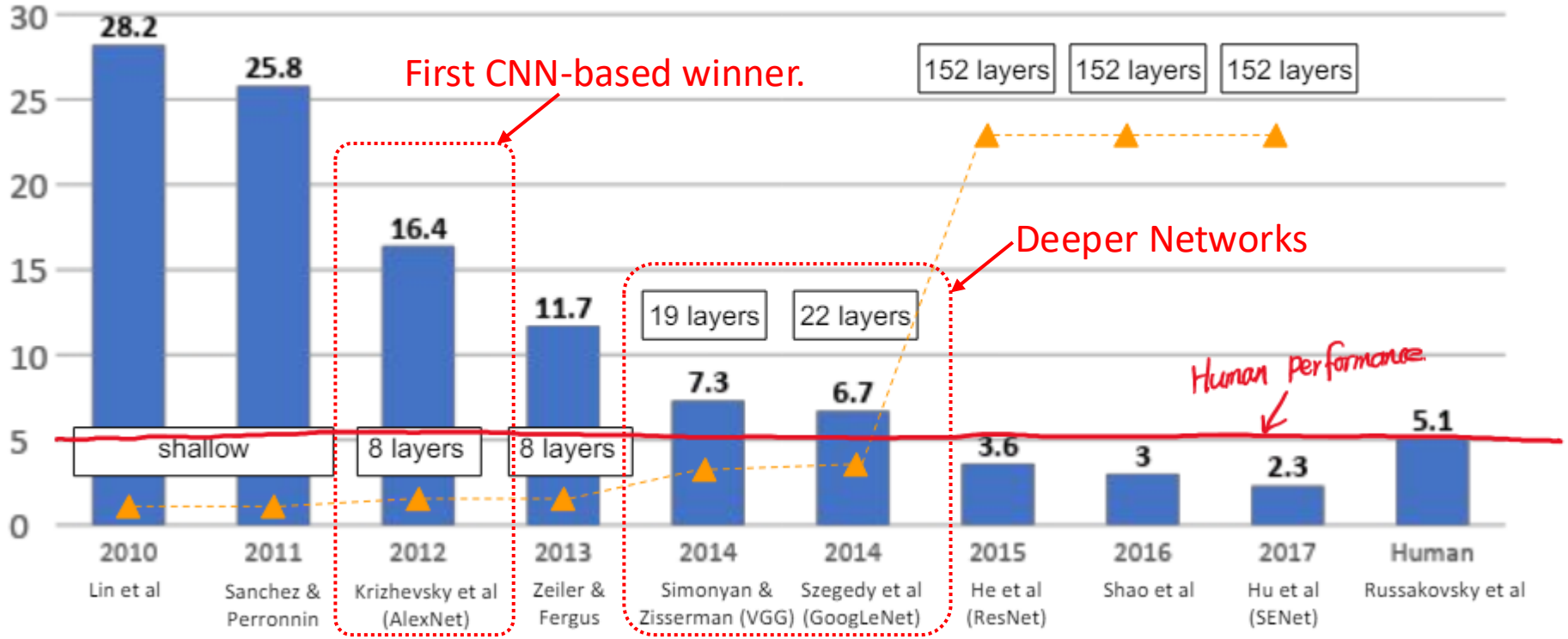
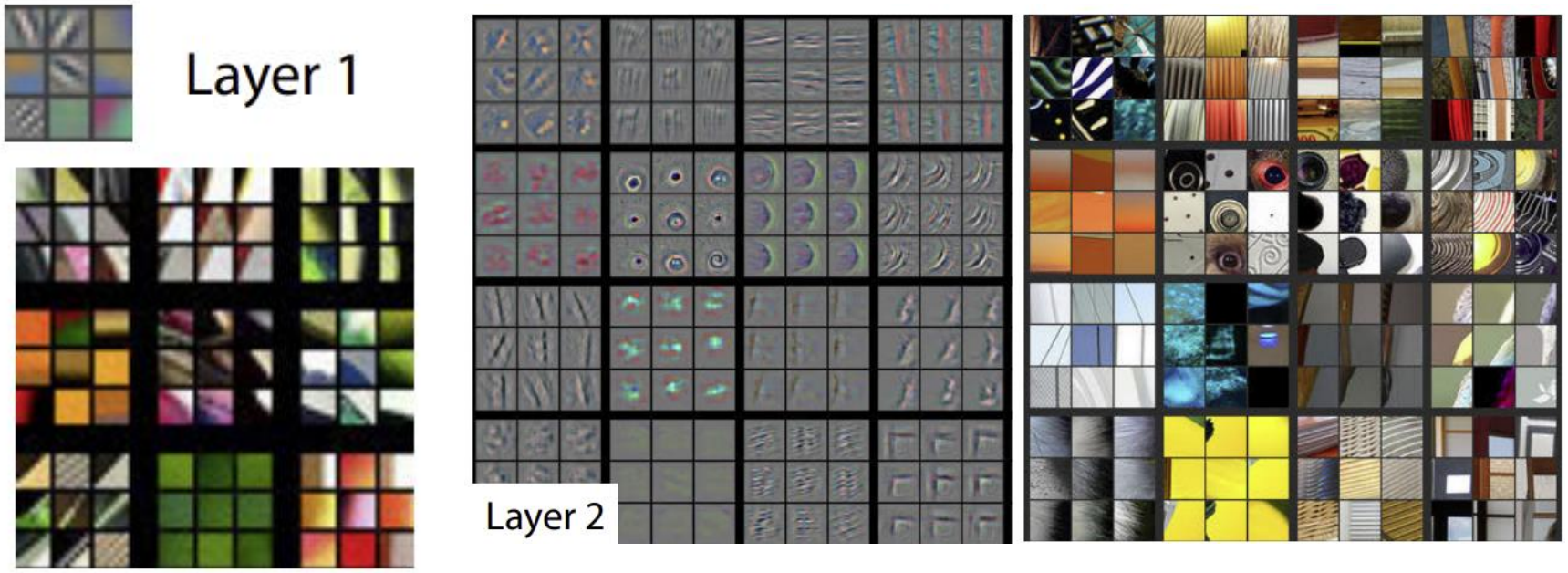


Image from Stanford cs231n

Unbox the Black-box: Visualization

- Visualization through Deconvolutional Network (Deconvnet) [Zeiler and Fergus, ECCV 2014]

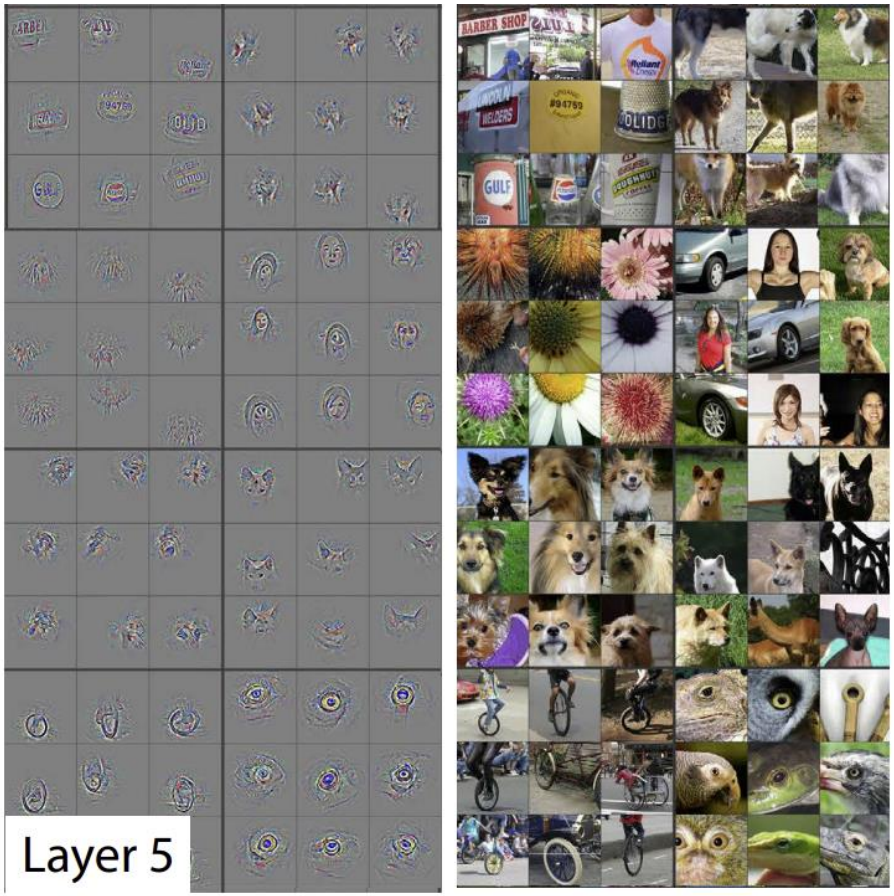


Visualization of the layerwise strongest activating features in an input image for a trained CNN.

[Zeiler and Fergus, ECCV 2014]

Unbox the Black-box: Visualization (Deconvnet)

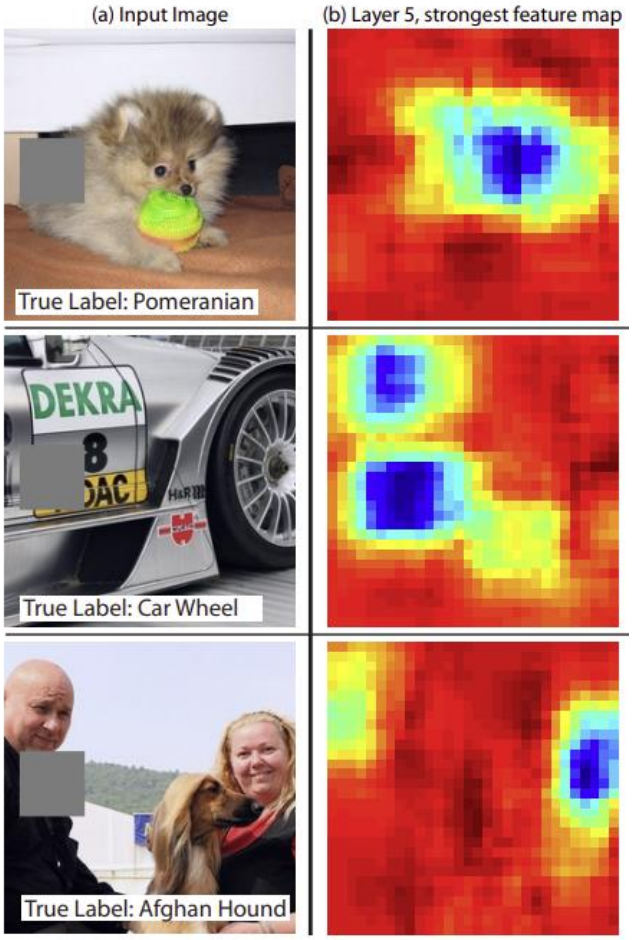
- Visualization through Deconvolutional Network (Deconvnet)



[Zeiler and Fergus, ECCV 2014]

Unbox the Black-box: Visualization (Heat Map)

- Visualization through Heat Map



When parts of the input image that are the strongest features for a class are occluded, the predicted score of the class drops.

[Zeiler and Fergus, ECCV 2014]

Unbox the Black-box: Visualization (CAM)

- Class Activation Mapping (CAM) [Zhou et al. CVPR 2016]

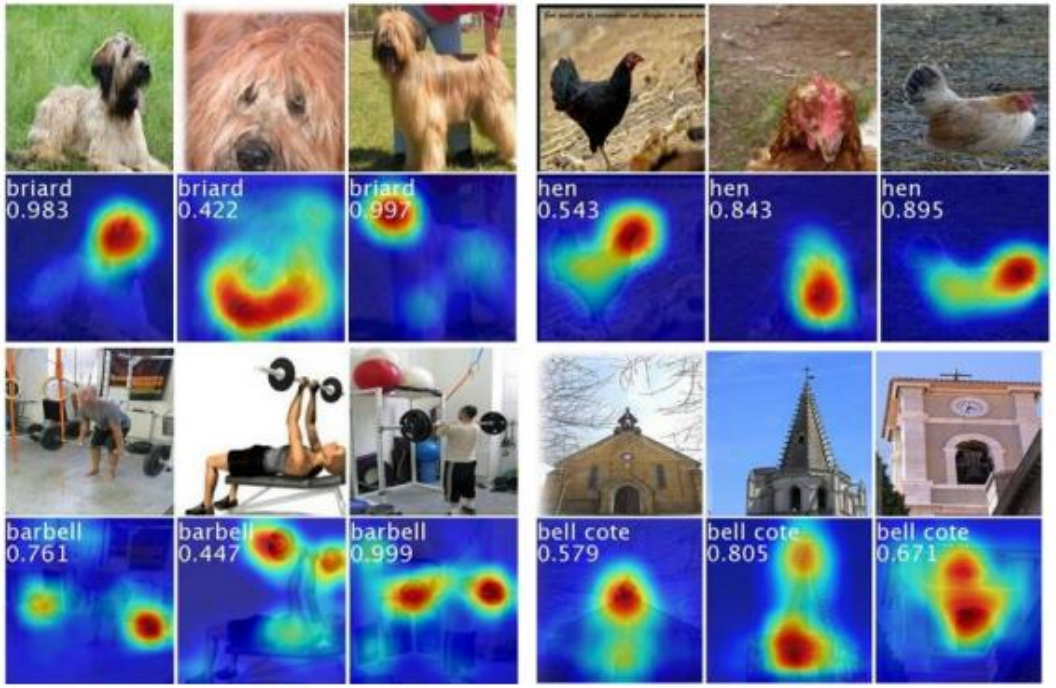
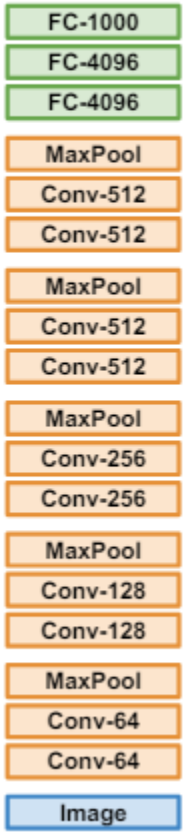


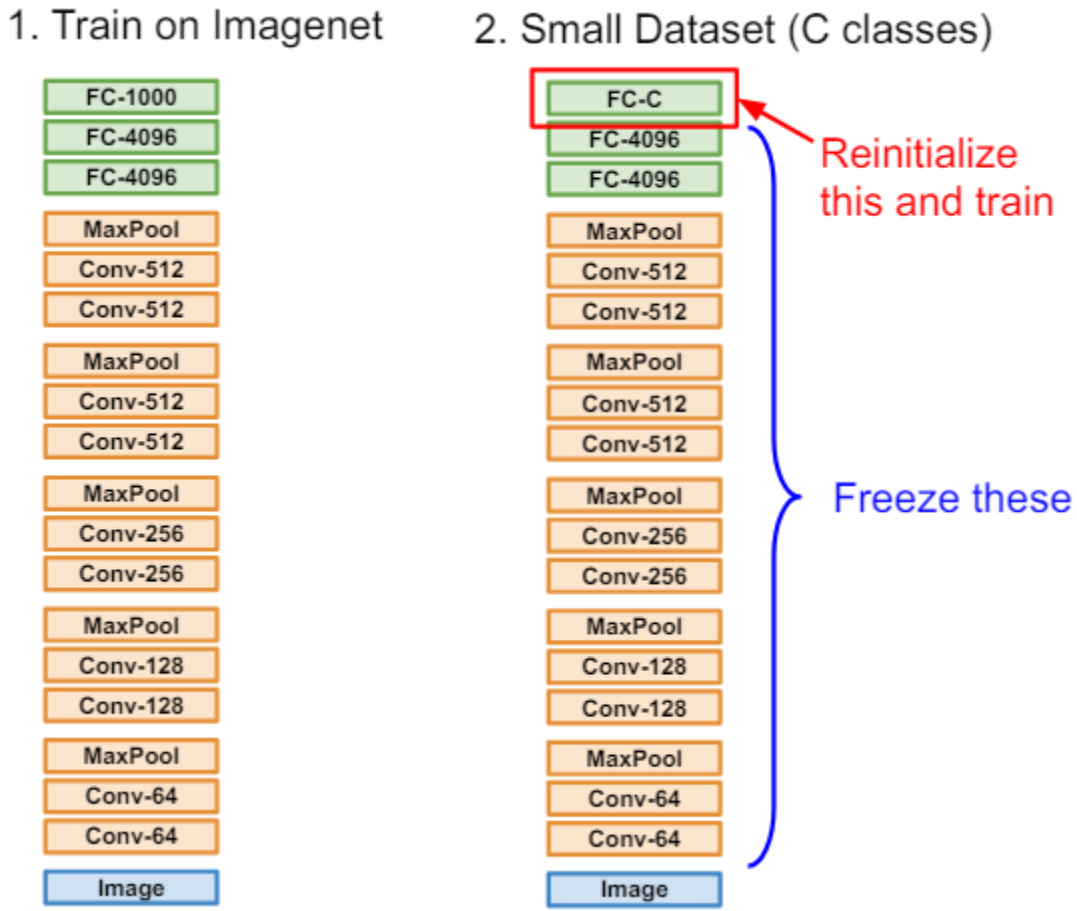
Figure 3. The CAMs of four classes from ILSVRC [20]. The maps highlight the discriminative image regions used for image classification e.g., the head of the animal for *briard* and *hen*, the plates in *barbell*, and the bell in *bell cote*.

Common Practice: Fine-tune with pre-trained model (1)

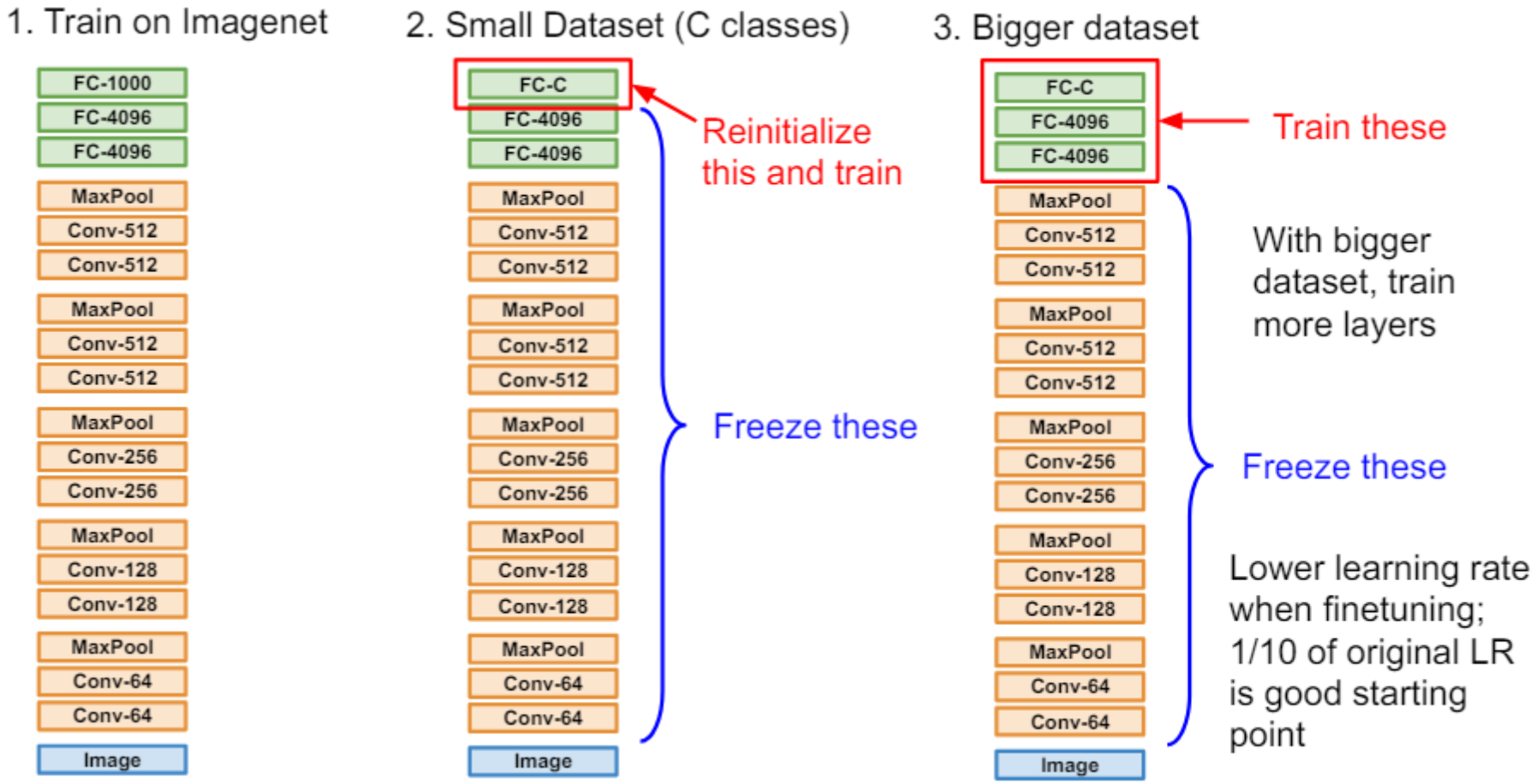
1. Train on Imagenet



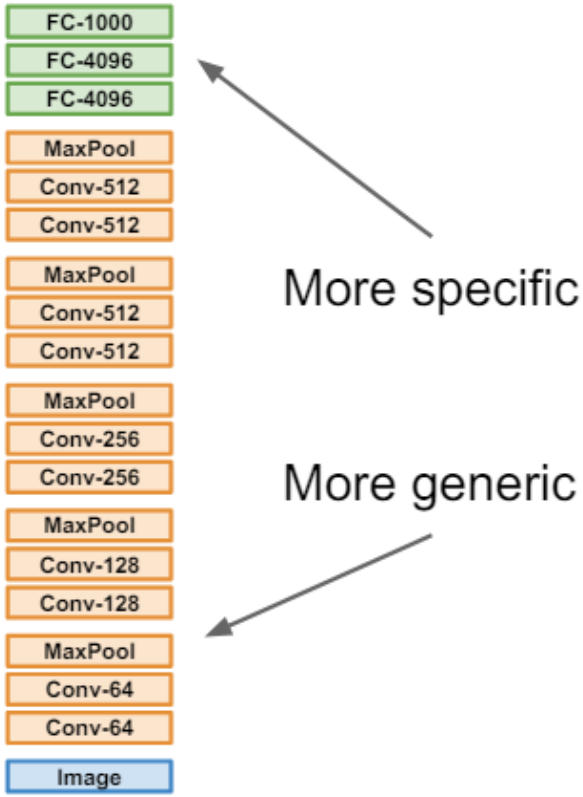
Common Practice: Fine-tune with pre-trained model (2)



Common Practice: Fine-tune with pre-trained model (3)



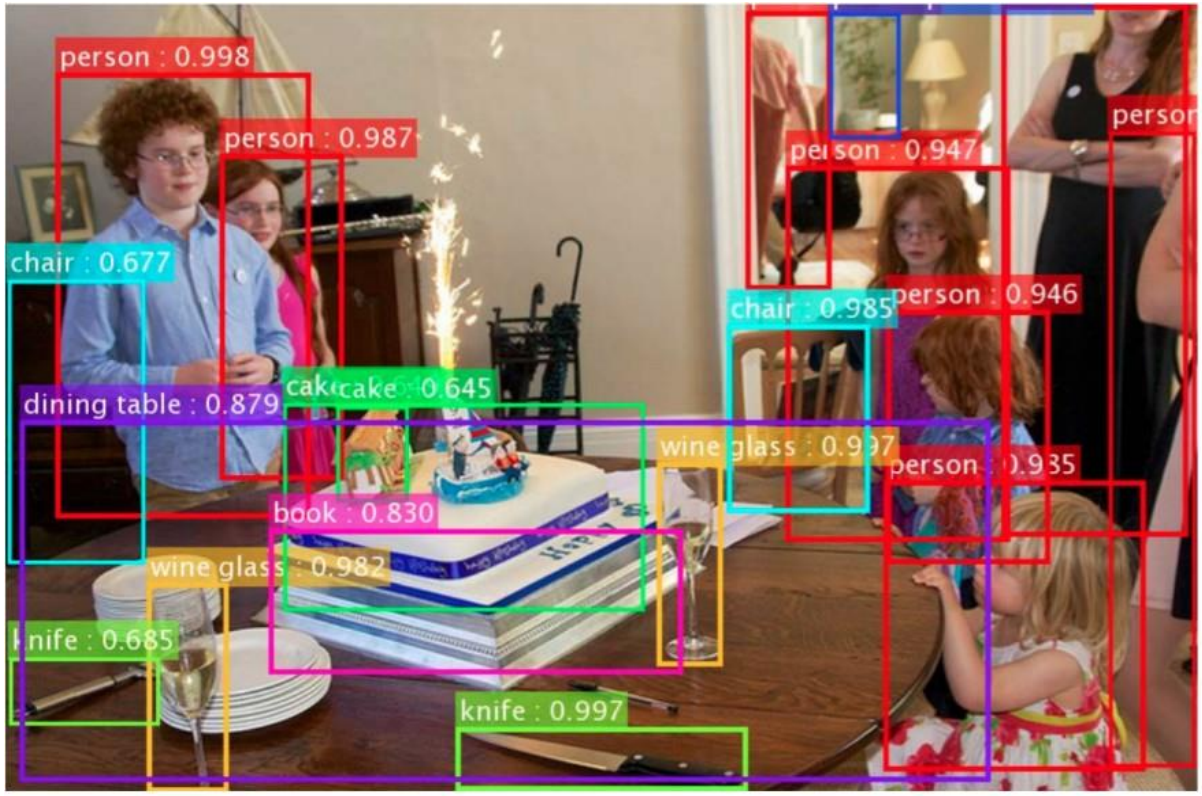
Common Practice: Fine-tune with pre-trained model (4)



	very similar dataset	**very different dataset**
very little data	Finetune linear classifier on top layer	You're in trouble… Try data augmentation / collect more data
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

Application Example:

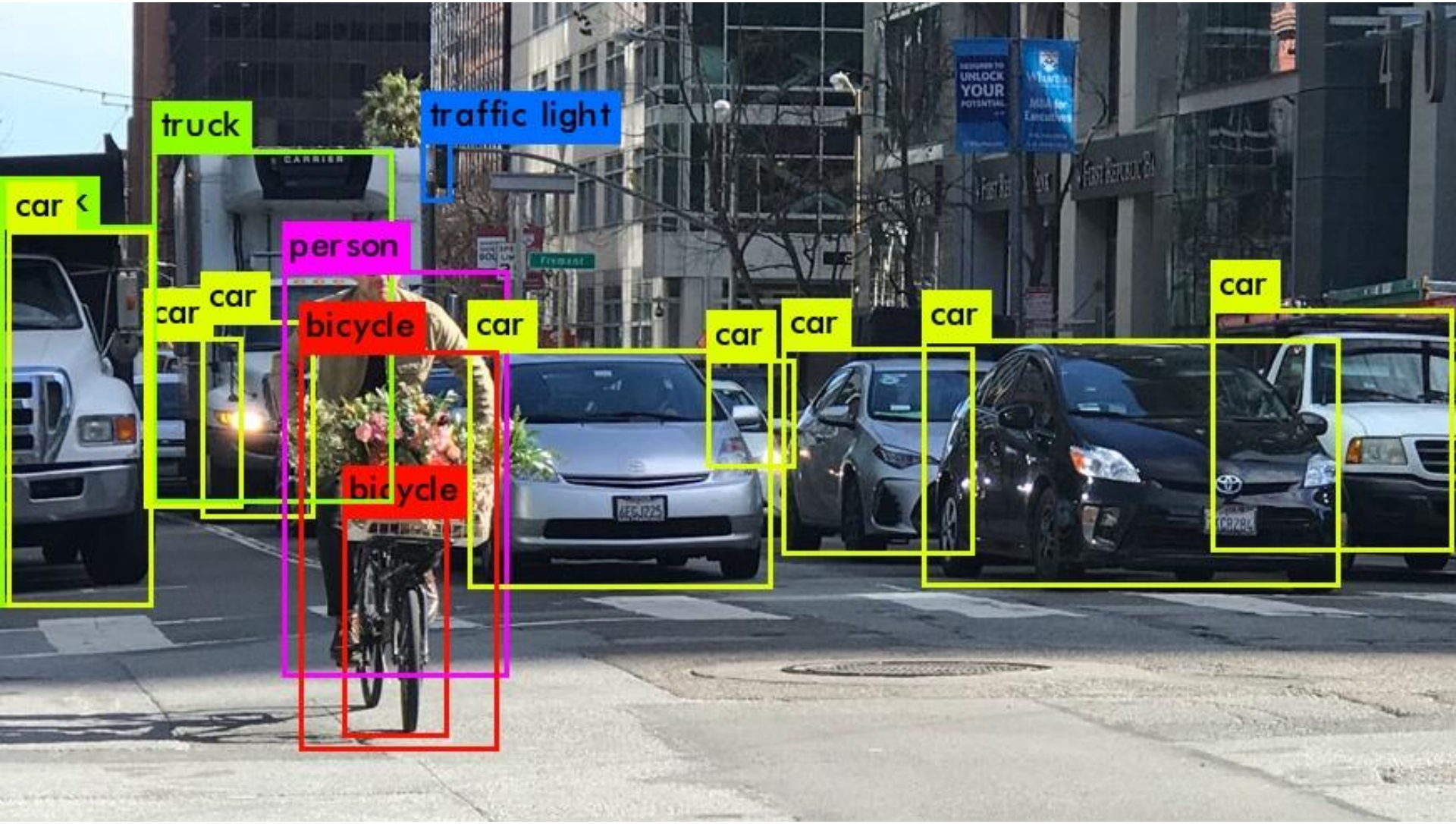
Object Detection



Slide: R. Liao, Paper: [He, K., Zhang, X., Ren, S. and Sun, J., 2015. Deep Residual Learning for Image Recognition. arXiv:1512.03385, 2016]

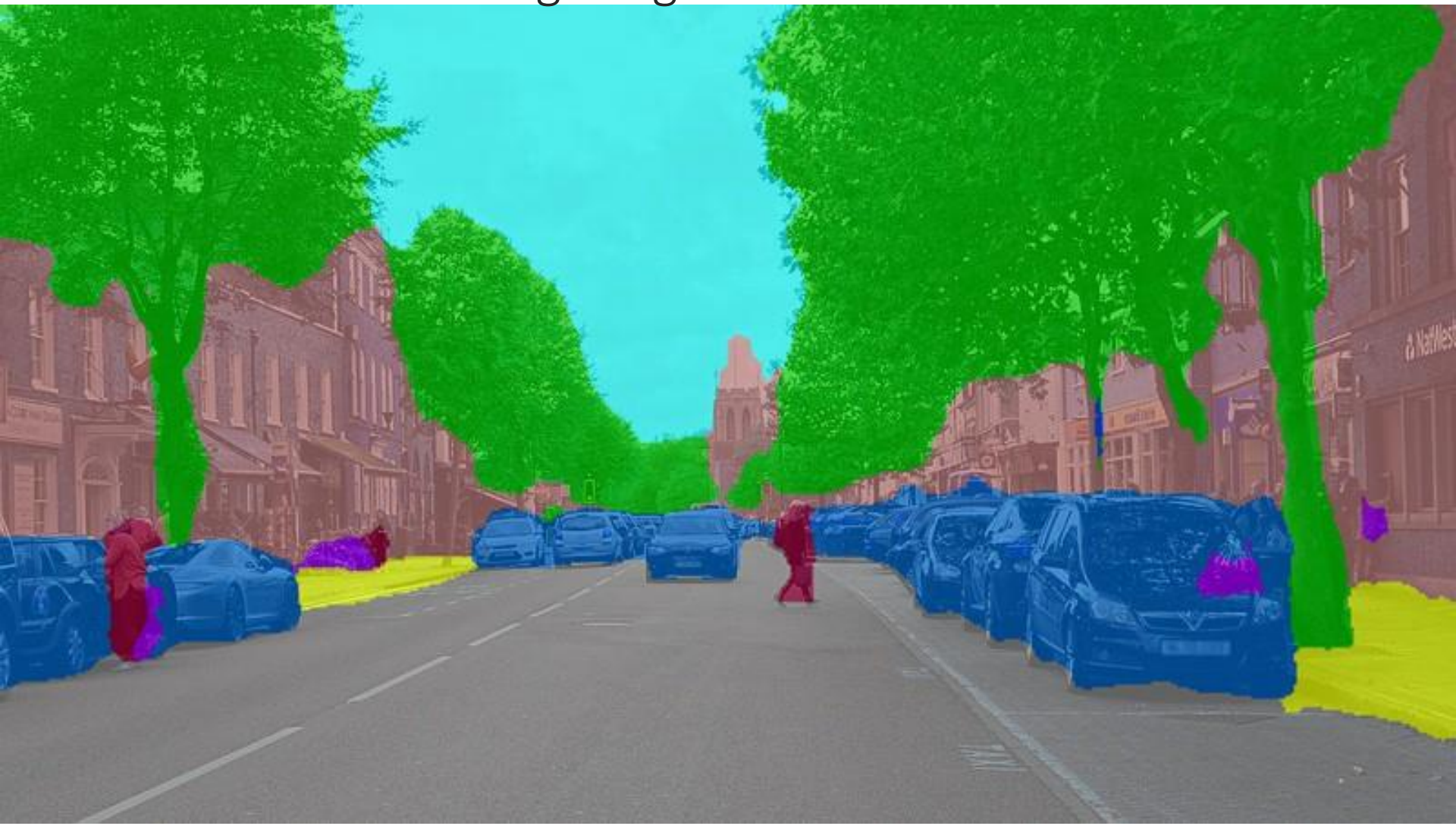
Application Example:

Autonomous Driving - Object Detection



Application Example:

Image Segmentation



Application Example:

Image Editing

Background Matting: The World is Your Green Screen

Soumyadip Sengupta
Vivek Jayaram
Brian Curless
Steve Seitz
Ira Kemelmacher-Shlizerman

University of Washington



Original Video

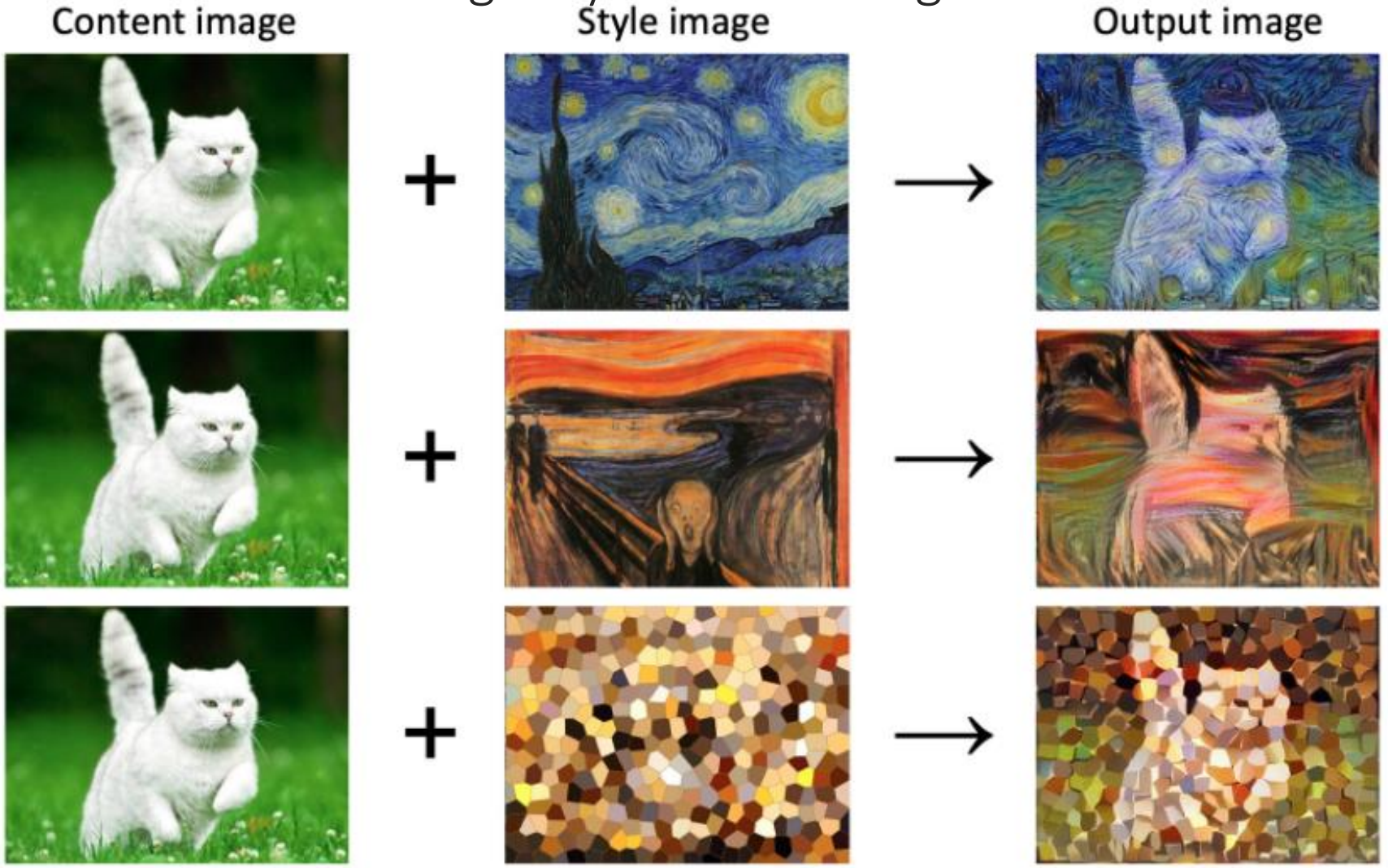


Replaced Background

https://grail.cs.washington.edu/projects/background-matting/

Application Example:

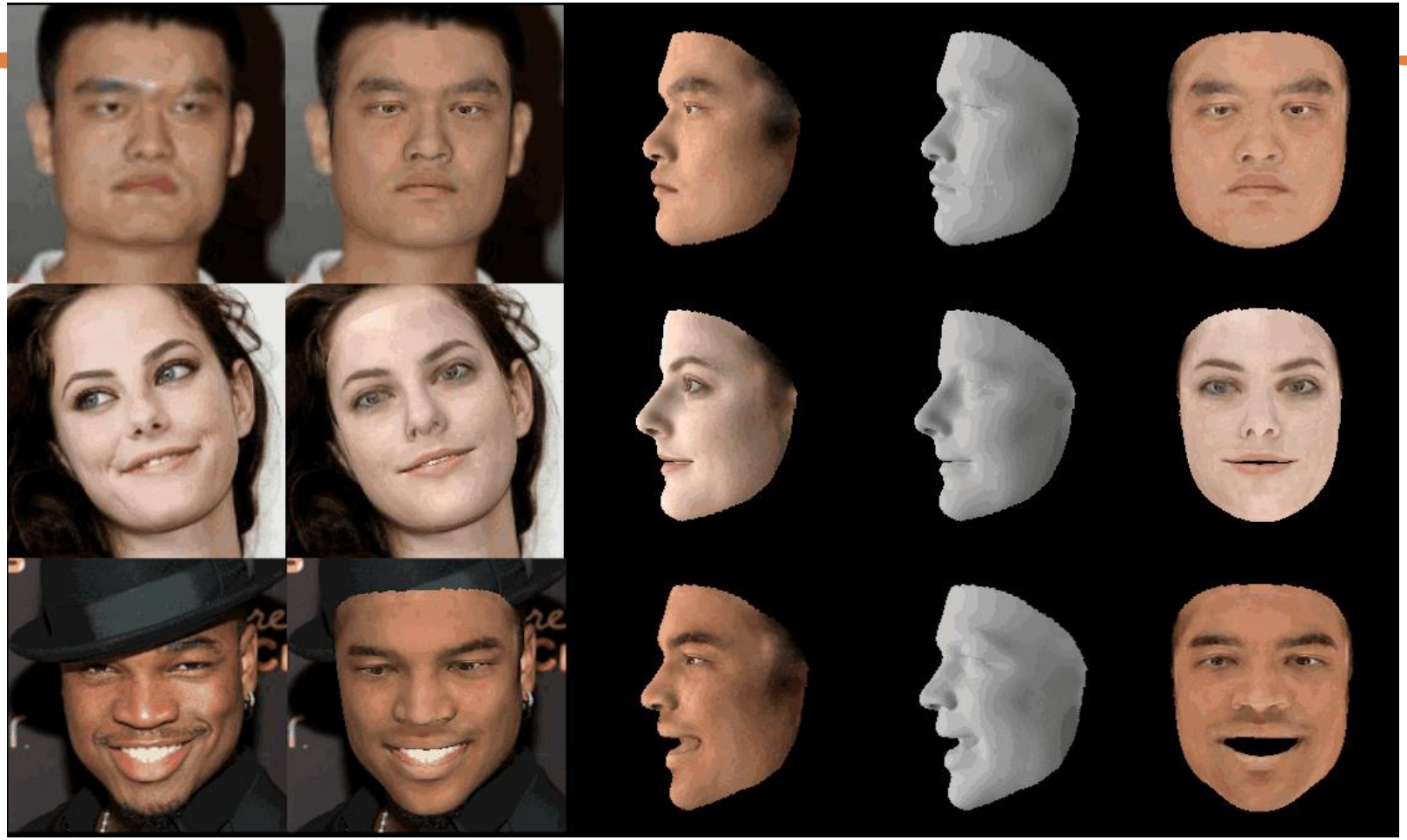
Image Style Transferring



https://xebia.com/blog/how-to-style-transfer-your-own-images/

Application Example:

3D Reconstruction



https://github.com/Microsoft/Deep3DFaceReconstruction

Application Example:

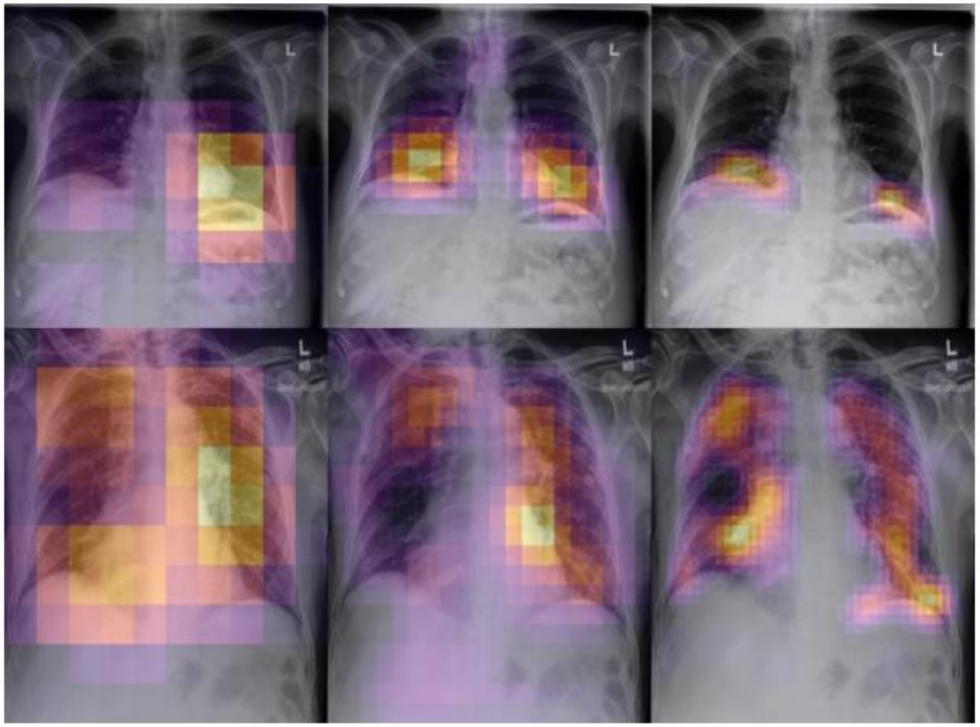
Multi-Object Tracking



https://github.com/CMU-Perceptual-Computing-Lab/openpose

Application Example:

Medical Image Analysis

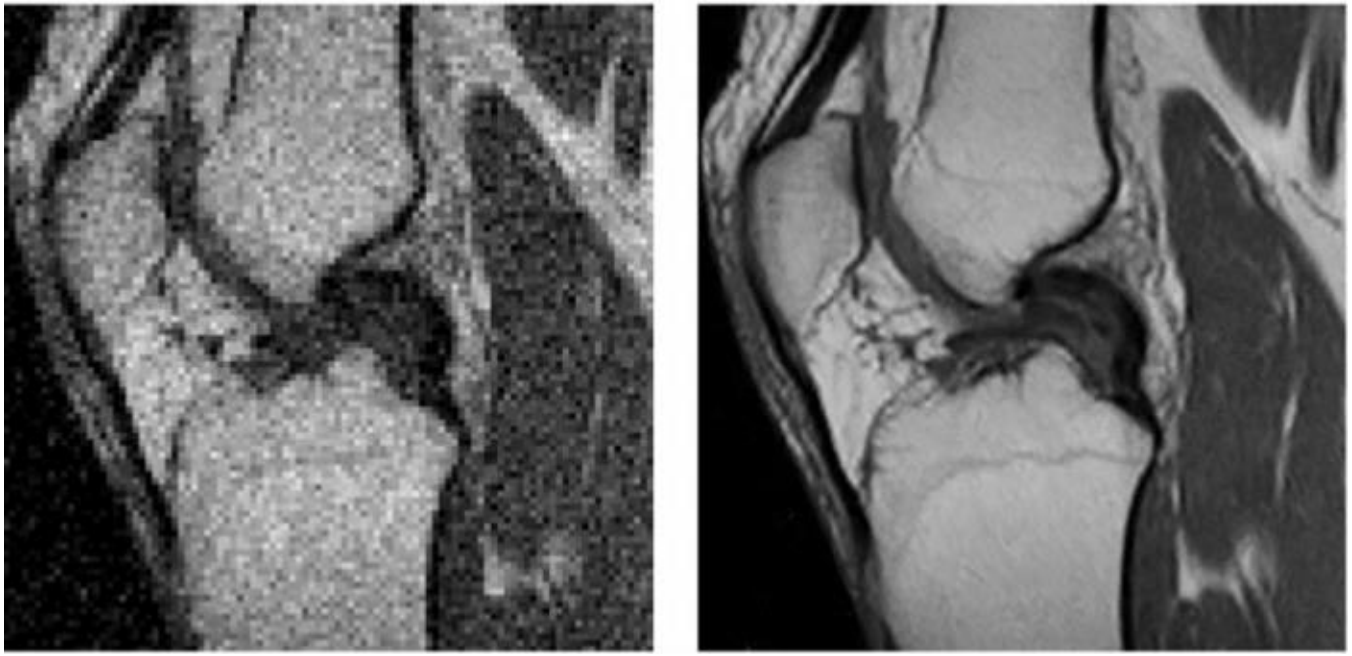


Deep Learning models for chest X-ray analysis

Alexander Ke, William Ellsworth, Oishi Banerjee, Andrew Y Ng, and Pranav Rajpurkar. 2021. CheXtransfer: performance and parameter efficiency of ImageNet models for chest X-Ray interpretation. In Proceedings of the Conference on Health, Inference, and Learning. 116–124

Application Example:

Medical Image Enhancement



left — Original LR Knee scan; right — SR image

Yue L, Shen H, Li J, Yuan Q, Zhang H, Zhang L. Image super-resolution: The techniques, applications, and future. Signal Processing. 2016 Nov 1;128:389-408.

Application Example:

Image Generation



https://zapier.com/blog/best-ai-image-generator/

Midjourney

An impressionist oil painting of a Canadian man riding a moose through a forest of maple trees

To Conclude:

Advantages

- Superior performance
- Wide range of Application
- End-to-end structure eliminates the need of feature engineering
- Efficient handling of data
- etc

Challenges:

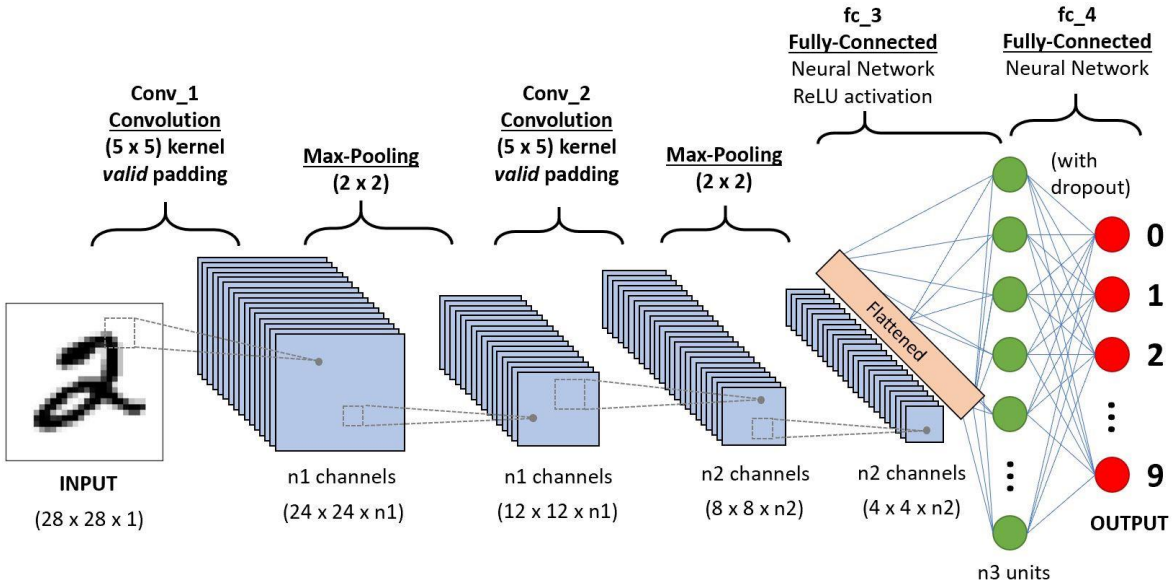
- Black-box
- Interpretability
- Fairness
- Privacy
- Mis-use

After-Class Questions (1)

- Q1: Explain the key difference between a **fully connected neural network** and a **convolutional neural network (CNN)**. Why are CNNs particularly suitable for image data?
- Q2: What is **weight sharing** in CNNs? Why does it significantly reduce the number of parameters?
- Q3: Explain the purpose of **pooling layers** (e.g., max pooling). What problem do they solve? What information is lost during pooling?
- Q4: What is a **feature map**? How does it evolve across layers in a CNN?

After-Class Questions (2)

- Q5: Given: Input: **28 × 28,** Filter: **3 × 3,** Stride: **1,** Padding: **1. (1)** Compute output size. (2) If there are **10 filters**, what is the output dimension?
- Q6: A convolutional layer has: 16 filters of size **3 × 3,** Input has **3 channels (RGB).** Compute the **total number of parameters.**
- Q7: Assume n1=16, n2=32, n3=4x4xn2, compute the total number of parameters in this CNN model.



After-Class Questions (3)

- Q8: A student reports that their CNN: Has very high training accuracy, but very low test accuracy. What is the likely issue? Suggest at least **three solutions.**

- Q9: CNNs are often described as having **translation invariance**. (1) What does this mean? (2) Which components of CNN contribute to this property?